

Digital Image Processing

Assignment - 1

September 7, 2026

Name: Harsh Kapoor
Roll No.: 2023112004
Email: harsh.kapoor@research.iiit.ac.in
CPU: 11th Gen Intel Core i5, 2.40GHz
RAM: 8GB DDR4
NumPy version: 2.2.6

1 Convolution

Data

The image is a 2048×2048 grayscale photograph



Figure 1: Data Image for P1

1.1 Correctness

a) Max error:

The maximum absolute errors are negligible, constrained only by 64-bit floating-point precision, we see that it is in the order of magnitude of 10^{-13} :

Implementation	Maximum Absolute Error
conv2d_loops	1.990×10^{-13}
conv2d_taps	1.990×10^{-13}
conv2d_im2col	1.990×10^{-13}
conv2d_fft	2.274×10^{-13}

b) Asymmetric Kernel Test:

To verify true convolution rather than correlation, an asymmetric kernel was tested. The maximum absolute errors were:

- Our convolution vs. SciPy convolution: 0.0
- Our convolution vs. SciPy correlation: 3422.0

This discrepancy against the correlation function proves that the kernel is being properly inverted (flipped) in our implementations.

c) Runtime and Extrapolation:

The naive nested-loop implementation (`conv2d_loops`) was timed on a 128×128 crop with a $k = 7$ kernel, averaging 0.2104 seconds. Extrapolating this to the full 2048×2048 image with a $k = 15$ kernel requires scaling by the ratio of both dimensions and kernel sizes:

$$\text{Scaling Factor} = \frac{2048^2 \times 15^2}{128^2 \times 7^2} \approx 1175.51$$

Applying this scaling factor yields an estimated runtime of 247.37 seconds (approximately 4 minutes). We are not asked to run the full-size version because pure Python nested loops incur massive interpreter overhead. Waiting over 4 minutes for a single basic image operation is entirely impractical for real-world image processing, demonstrating the necessity of vectorization and efficient memory access patterns.

1.2 Kernel Rank

Numerical Rank and Threshold

The numerical rank of each 15×15 kernel was computed by counting the number of singular values strictly greater than a threshold of 10^{-10} . This explicit threshold is necessary because finite-precision floating-point arithmetic introduces non-zero singular values for mathematically zero components; without it, numerical noise would inflate the rank of low-rank kernels.

Kernel ($k = 15$)	Numerical Rank
box	1
gaussian	1
sobel3	1
log	2
log_dc_removed	3
motion_0deg	1
motion_45deg	15
disk	5
random	15

Table 1: Numerical rank of kernels

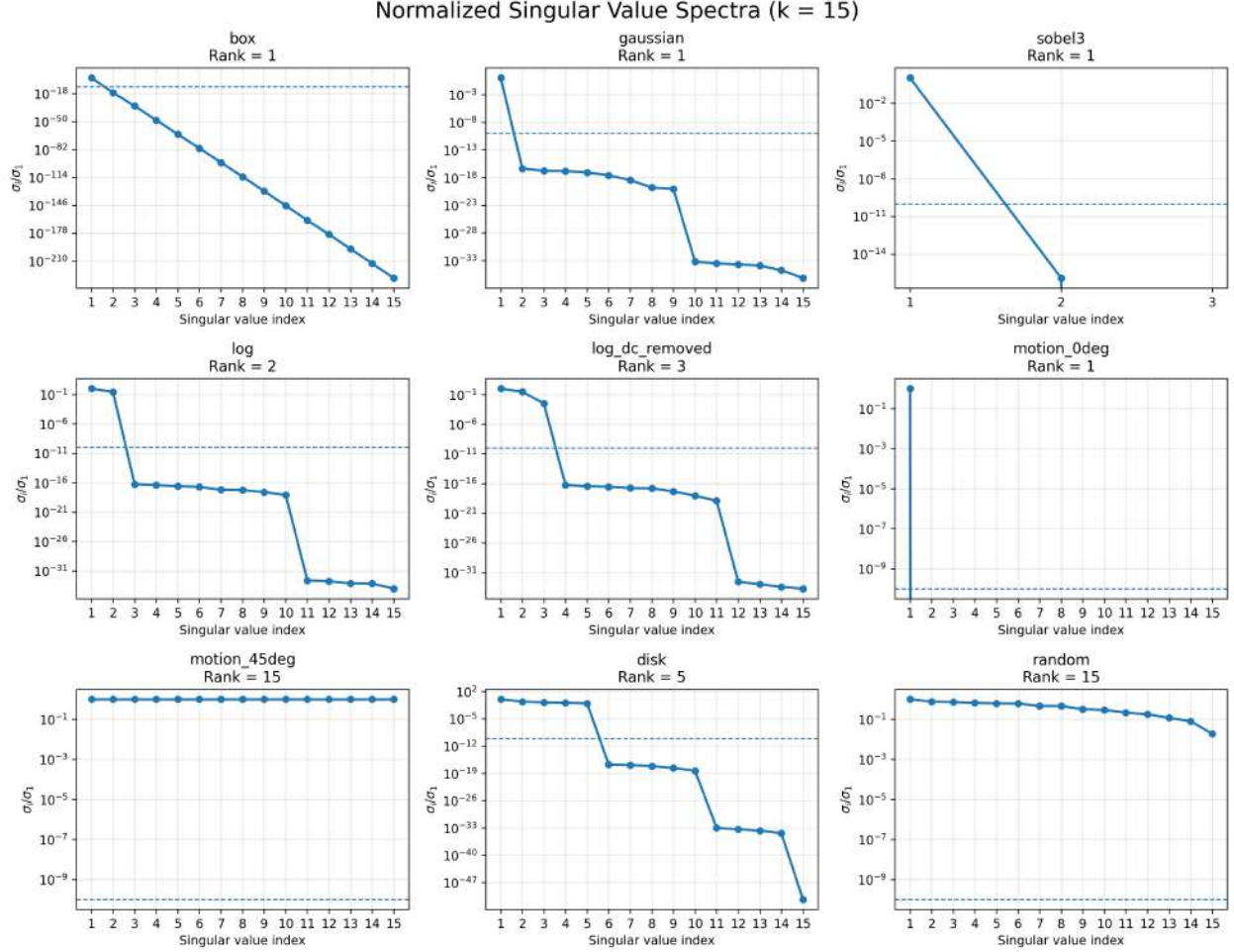


Figure 2: Spectra of Singular Values

a) Raw LoG Rank Derivation

The continuous Laplacian of Gaussian (LoG) function is defined as :

$$LoG(x, y) = \frac{r^2 - 2\sigma^2}{\sigma^4} \exp\left(\frac{-r^2}{2\sigma^2}\right)$$

Since $r^2 = x^2 + y^2$, we can expand the equation and separate the spatial coordinates:

$$LoG(x, y) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} \exp\left(-\frac{x^2}{2\sigma^2}\right) \exp\left(-\frac{y^2}{2\sigma^2}\right)$$

Grouping the terms by coordinate yields:

$$LoG(x, y) = \left[\frac{x^2 - \sigma^2}{\sigma^4} \exp\left(-\frac{x^2}{2\sigma^2}\right) \right] \exp\left(-\frac{y^2}{2\sigma^2}\right) + \left[\frac{y^2 - \sigma^2}{\sigma^4} \exp\left(-\frac{y^2}{2\sigma^2}\right) \right] \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

This demonstrates that the continuous LoG function is exactly the sum of two distinct, linearly independent separable functions. In matrix form, a separable 2D function forms a rank-1 matrix (the outer product of a column and row vector). Thus, the sampled LoG matrix is the sum of two rank-1 matrices, giving it an exact rank of 2.

b) Mean-Subtracted LoG Rank

Subtracting the mean from the LoG matrix is mathematically equivalent to adding a uniform matrix C (where every element is $-mean$). A uniform matrix is defined by a single constant scalar and is strictly rank-1. The general upper bound for matrix addition is $rank(A + B) \leq rank(A) + rank(B)$. Because the flat rank-1 matrix C is linearly independent of the two basis vectors that form the rank-2 LoG matrix, they do not share a null space. Therefore, equality holds ($2 + 1 = 3$), raising the final numerical rank exactly to 3.

c) Motion Blur Matrix Structures

The 0° motion blur kernel consists of a single horizontal line of non-zero values across the center row, with all other rows being zero. Since all rows are scalar multiples of each other, the row space is 1-dimensional, making it rank 1. Conversely, the 45° motion blur kernel places values strictly along the main diagonal. This forms a diagonal matrix where every row and column is linearly independent. Consequently, the basis vectors span the entire space, resulting in full rank (15).

1.3 Separability and low rank

a) Low-Rank Approximation Performance

The separable convolution and truncated SVD low-rank convolution functions were evaluated on a 512×512 image crop using $k = 21$ kernels. The Peak Signal-to-Noise Ratio (PSNR) was measured against the exact 2D convolution for the disk, LoG, 45° motion, and random kernels across all possible ranks $r \in [1, 21]$. The generated plot maps how rapidly the approximation converges to the exact output for each spatial filter as r increases.

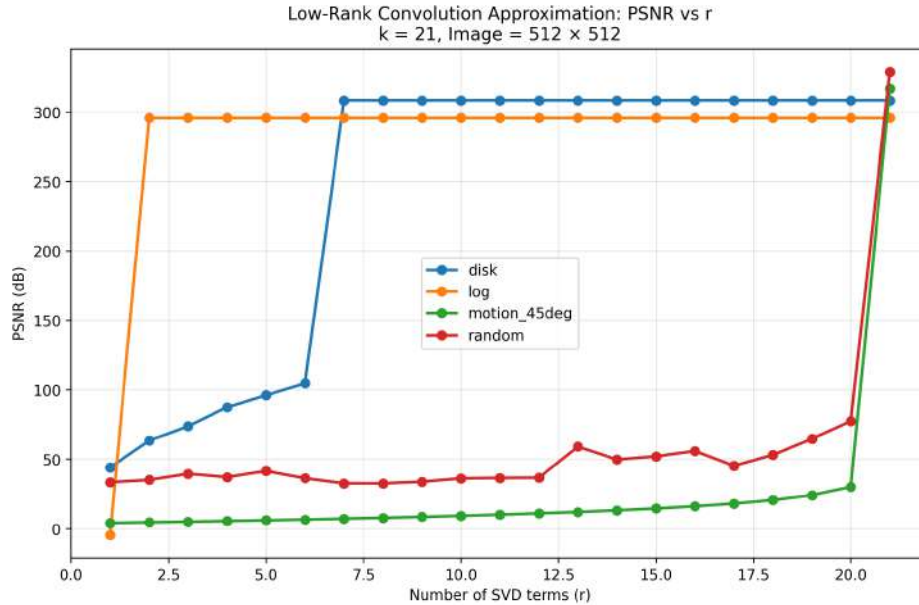


Figure 3: PSNR vs r

b) Spectral Distinctions and PSNR Prediction

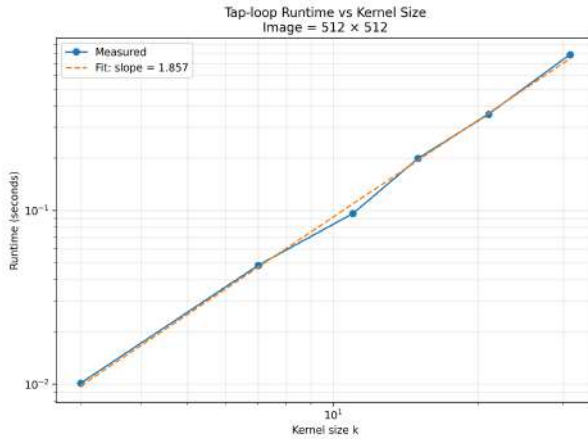
Both the disk and 45° motion kernels are non-separable ($\text{rank} > 1$), yet their low-rank approximations behave entirely differently. The disk kernel achieves a high fidelity (PSNR > 73 dB) with just $r = 3$ terms, whereas the diagonal motion kernel requires all $k = 21$ terms to achieve exact recovery.

This distinction is fundamentally driven by the distribution of their singular values, making it entirely predictable from the singular value spectrum alone:

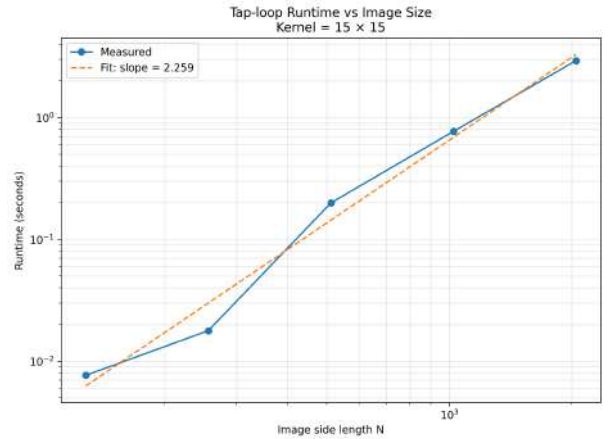
- **Disk Kernel (Steep Decay):** Its singular value spectrum exhibits a massive, precipitous drop after the first few components. Because the leading singular values ($\sigma_1, \sigma_2, \sigma_3$) are magnitudes larger than the rest, they capture the vast majority of the matrix’s energy and spatial structure. A truncated SVD at $r = 3$ reconstructs almost the exact kernel, leading to near-perfect fidelity.
- **Motion- 45° Kernel (Flat Spectrum):** Its singular value spectrum is perfectly flat ($\sigma_1 = \sigma_2 = \dots = \sigma_k$). Every basis vector contributes equally to the matrix structure. Because the energy is distributed uniformly, truncating the SVD at any $r < k$ discards critical structural information, resulting in poor PSNR until the final term is included.

Ultimately, a steeply decaying spectrum dictates that a kernel is highly compressible via low-rank approximation, while a flat spectrum indicates that dimensionality reduction will fail.

1.4 Runtime analysis



(a) Runtime vs k at 512^2



(b) Runtime vs N at $k = 15$

Figure 4: Runtime scaling of the tap-loop convolution.

Patch-Matrix Memory Derivation

The `im2col` algorithm extracts a $k \times k$ patch for every pixel in the $H \times W$ image. This forms a patch matrix of size $(H \cdot W) \times (k \cdot k)$. Therefore, the total number of elements is $N^2 k^2$.

Assuming double-precision 64-bit floats (`float64`), which require 8 bytes per element, the memory formula in bytes is:

$$\text{Memory (bytes)} = 8 \cdot N^2 \cdot k^2$$

N	k	Patch Elements	Memory (MB)	Memory (GiB)
128	15	3,686,400	29.49	0.027
256	15	14,745,600	117.96	0.110
512	15	58,982,400	471.86	0.439
1024	15	235,929,600	1887.44	1.758
2048	15	943,718,400	7549.75	7.031

Table 2: Peak `im2col` patch-matrix memory requirements.

At $N = 2048$ and $k = 15$, the patch matrix demands over 7 GiB of contiguous RAM. On memory-constrained systems, attempting to allocate this matrix will result in a **MemoryError** (OOM crash) or severe thrashing as the operating system relies on swap space.

a) Missing Exponent in k Scaling

Theory dictates that a 2D convolution’s arithmetic complexity scales as $O(k^2)$. However, the empirically fitted slope from the log-log plot is 1.8573. The missing exponent is absorbed by fixed execution overheads. In Python, looping overhead, array slicing, bounds checking, and function calls consume a baseline amount of time that scales linearly (or remains constant) regardless of k . For smaller kernels, this non-arithmetic overhead constitutes a larger percentage of the total execution time, diluting the pure quadratic arithmetic scaling.

b) `im2col` vs. Tap Loop

On this machine, `im2col` performs significantly worse than the tap loop for large images (taking 30.95 seconds versus 2.86 seconds at $N = 2048$).

N	Tap Loop (s)	<code>im2col</code> (s)
128	0.0081	0.0156
256	0.0194	0.0452
512	0.1890	0.1692
1024	0.7484	0.7726
2048	2.8618	30.9492

The `im2col` method wins only when running on highly parallelized hardware, such as GPUs, paired with highly optimized BLAS (Basic Linear Algebra Subprograms) libraries. In those environments, transforming the operation into a single massive matrix multiplication masks the memory duplication cost through sheer floating-point throughput. Those conditions do not hold here; running pure NumPy on a CPU shifts the bottleneck entirely to memory bandwidth. The CPU wastes massive amounts of time allocating and moving 7 GiB of duplicated data in and out of RAM, completely negating any vectorization benefits.

c) Separable Speedup Shift

For a 15×15 kernel, separable convolution drops the operation count from k^2 (225) to $2k$ (30). The theoretical arithmetic speedup is exactly $7.5\times$.

At $N = 512$, the measured speedup is $8.359\times$, which exceeds the theoretical arithmetic bound. This occurs because the smaller image and intermediary arrays fit comfortably within the CPU's fast L2/L3 cache, leading to highly efficient memory access.

At $N = 2048$, the speedup normalizes to $7.572\times$. A 2048×2048 `float64` array requires roughly 33 MB of memory, exceeding the capacity of most standard CPU caches. As a result, the processor is forced to continuously fetch data from slower main memory (RAM). The operation becomes memory-bound (bottlenecked by read/write speeds) rather than compute-bound, stripping away the super-linear cache efficiency and bringing the real-world speedup closer to the theoretical arithmetic limit.

2 Bit-plane slicing and Steganography

Data

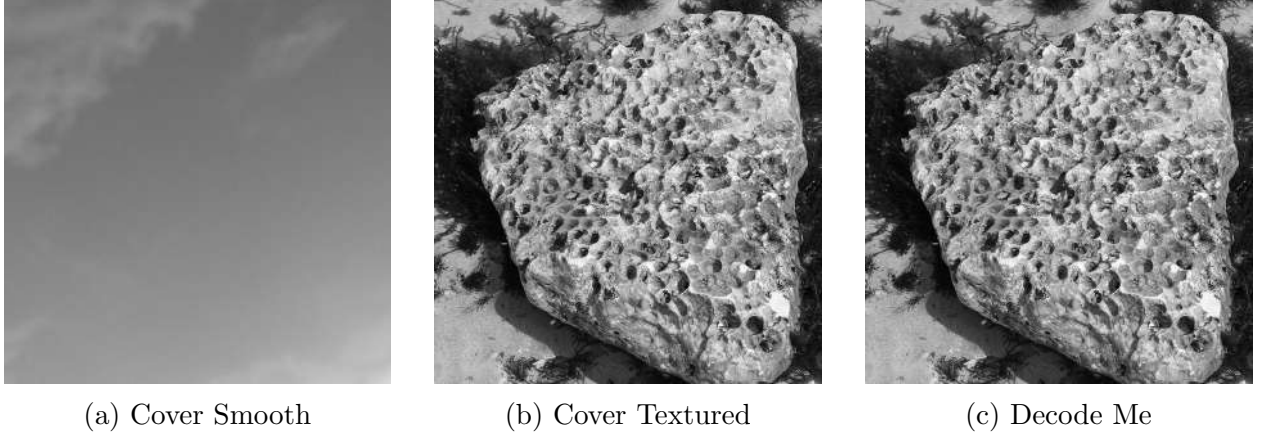


Figure 5: Images used for the steganography experiment.

2.1 Bit-Plane Decomposition

The supplied textured cover is a 1024×1024 8-bit grayscale image. Each pixel can therefore be represented using eight binary bit planes. For an image intensity I , the k -th bit plane is obtained as

$$B_k(I) = (I \gg k) \& 1, \quad k = 0, \dots, 7,$$

where B_0 is the least significant bit (LSB) and B_7 is the most significant bit (MSB). The implementation extracts all eight planes directly from the uint8 image.

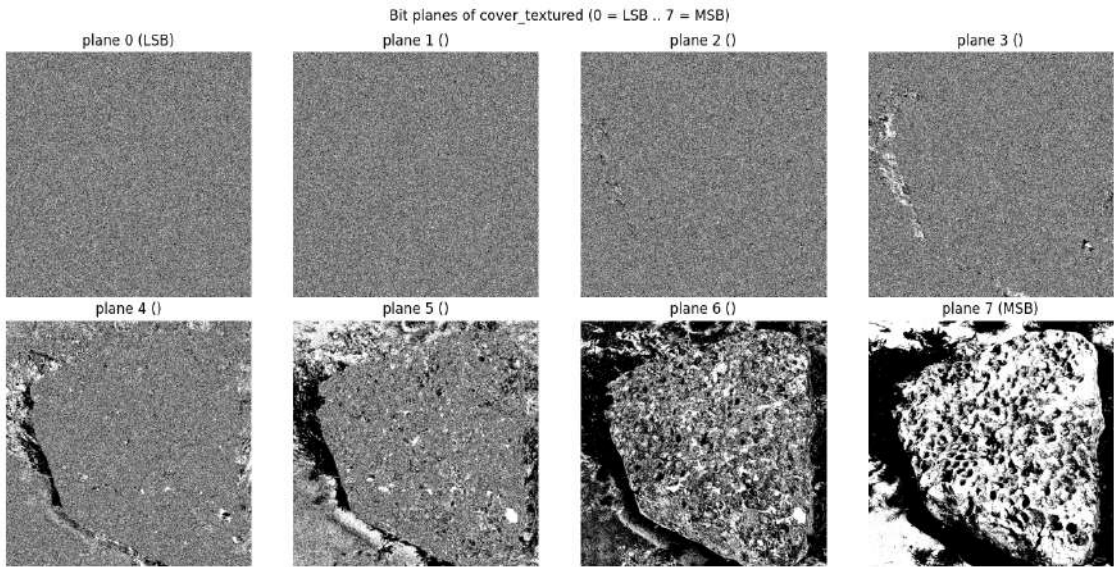


Figure 6: Eight bit planes of the textured cover, from the LSB (0) to the MSB (7).

The image was reconstructed using only the n most significant bit planes, for $n = 1, \dots, 8$. The reconstruction retains planes $7, 6, \dots, 8 - n$ and sets the remaining lower-order planes to zero. The resulting PSNR values are shown in Fig. 7.

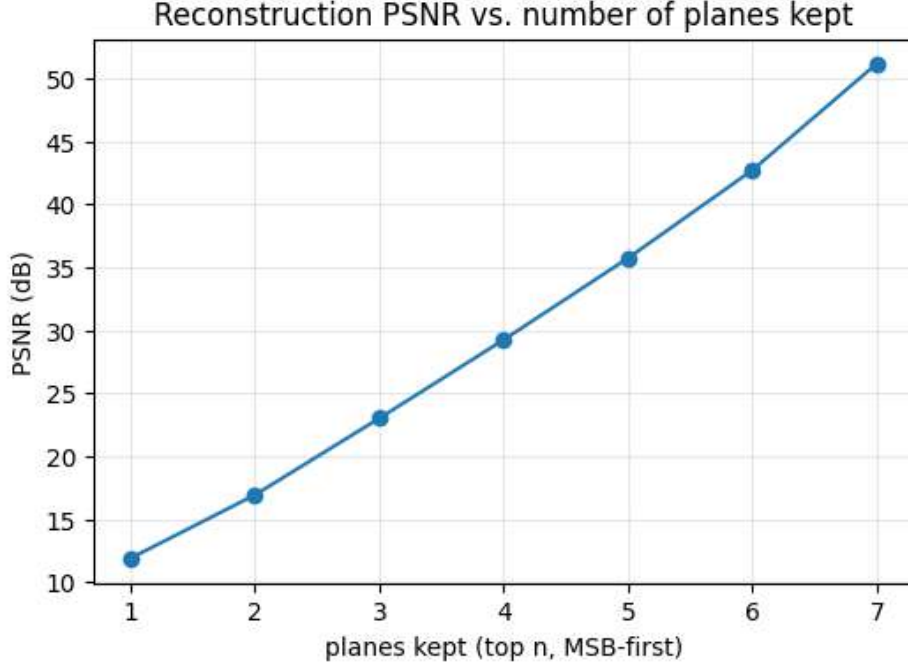


Figure 7: Reconstruction PSNR as a function of the number of retained most-significant bit planes.

The measured PSNR values are

Number of MSB planes	PSNR (dB)
1	11.891
2	16.918
3	22.999
4	29.227
5	35.707
6	42.692
7	51.140

Table 3: PSNR obtained by reconstructing the image using only the most-significant bit planes.

The mean PSNR improvement between consecutive finite values is

$$\boxed{6.542 \text{ dB}}.$$

The approximately 6 dB increase per additional bit plane follows from the binary quantisation structure. Adding one more bit reduces the quantisation step by approximately

a factor of two. Since MSE depends quadratically on the quantisation step, the MSE is reduced by approximately a factor of four, corresponding to

$$10 \log_{10}(4) = 20 \log_{10}(2) \approx 6.02 \text{ dB}.$$

Thus, the approximately 6 dB improvement is a consequence of binary representation and quantisation rather than a property specific to the particular image.

Binary and Gray-coded ramps. A linear intensity ramp was also represented using ordinary binary coding and Gray coding. Gray coding was generated using

$$G = I \oplus (I \gg 1),$$

where $\oplus$ denotes the bitwise XOR operation.

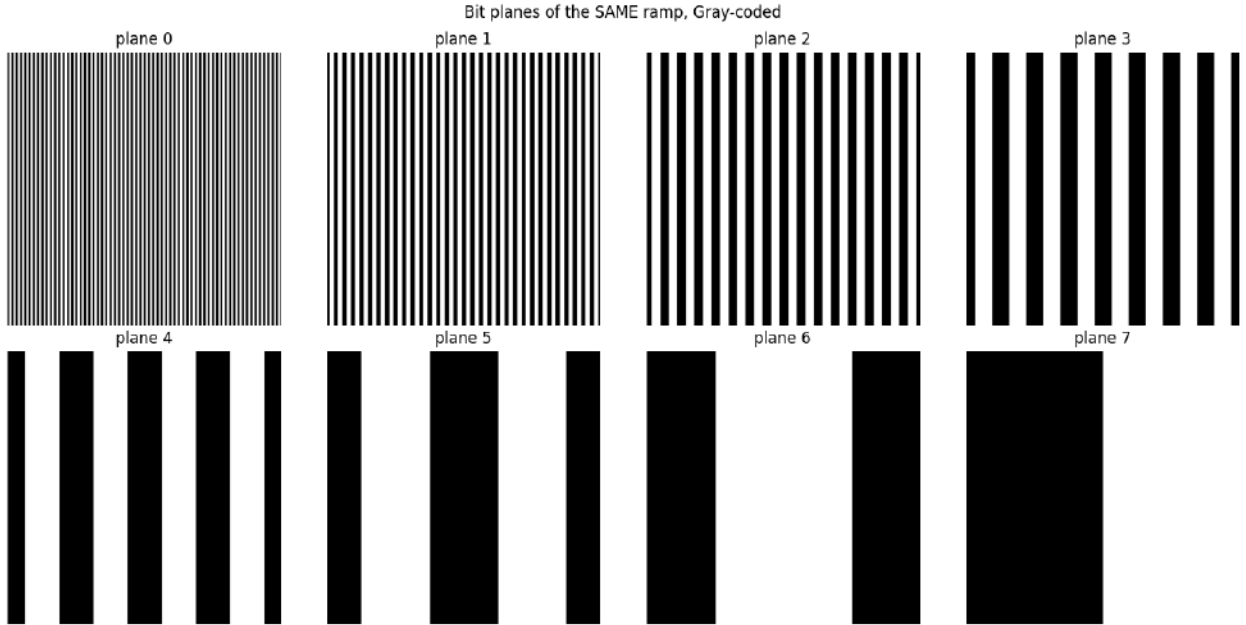


Figure 8: Bit-plane representation of a linear intensity ramp using Gray coding.

In ordinary binary coding, several bits can change simultaneously between consecutive intensity levels. For example, a binary transition such as $0111 \rightarrow 1000$ changes multiple bit planes at once, creating strong periodic structures in the individual bit planes.

Gray coding has the property that consecutive intensity values differ in only one bit. Consequently, the simultaneous transitions present in ordinary binary coding are avoided and the structure of the low-order bit planes is substantially changed.

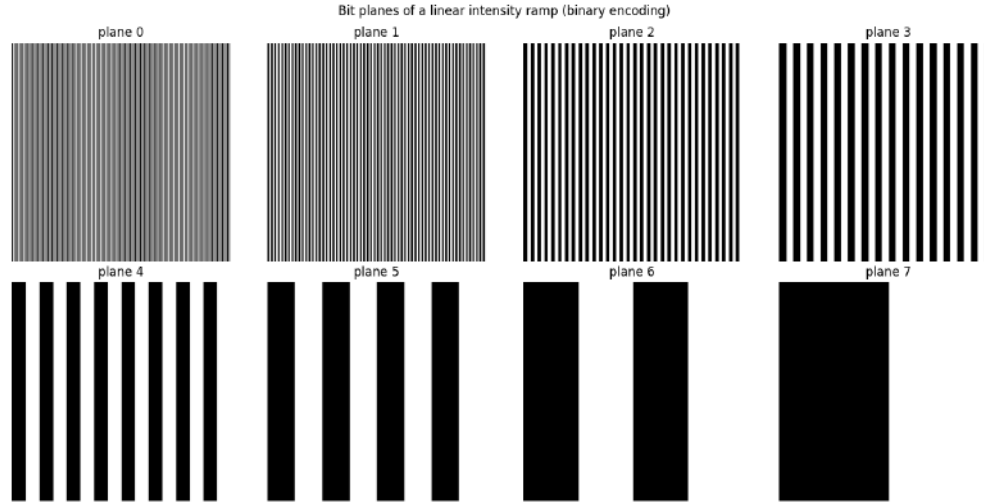


Figure 9: Bit-plane representation of a linear intensity ramp using binary coding.

2.2 LSB Steganography

The supplied stego image was decoded using the least significant bit of each pixel. Pixels were traversed in raster order. The first 24 bits were interpreted as an 8-bit synchronization marker followed by the payload height and width.

The recovered synchronization marker was

10101010.

The recovered payload dimensions were

64×64 .

Therefore, the payload contains

$$64 \times 64 = 4096$$

payload bits.

Recovered payload (64x64)

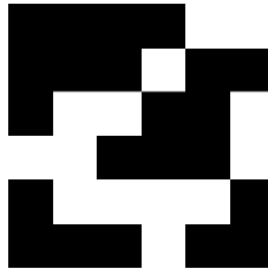


Figure 10: Payload recovered from the LSBs of the supplied stego image.

The extracted payload confirms that the LSB ordering and header interpretation are consistent with the supplied stego image.

The difference between the supplied stego image and the clean textured cover is shown in Fig. 11.

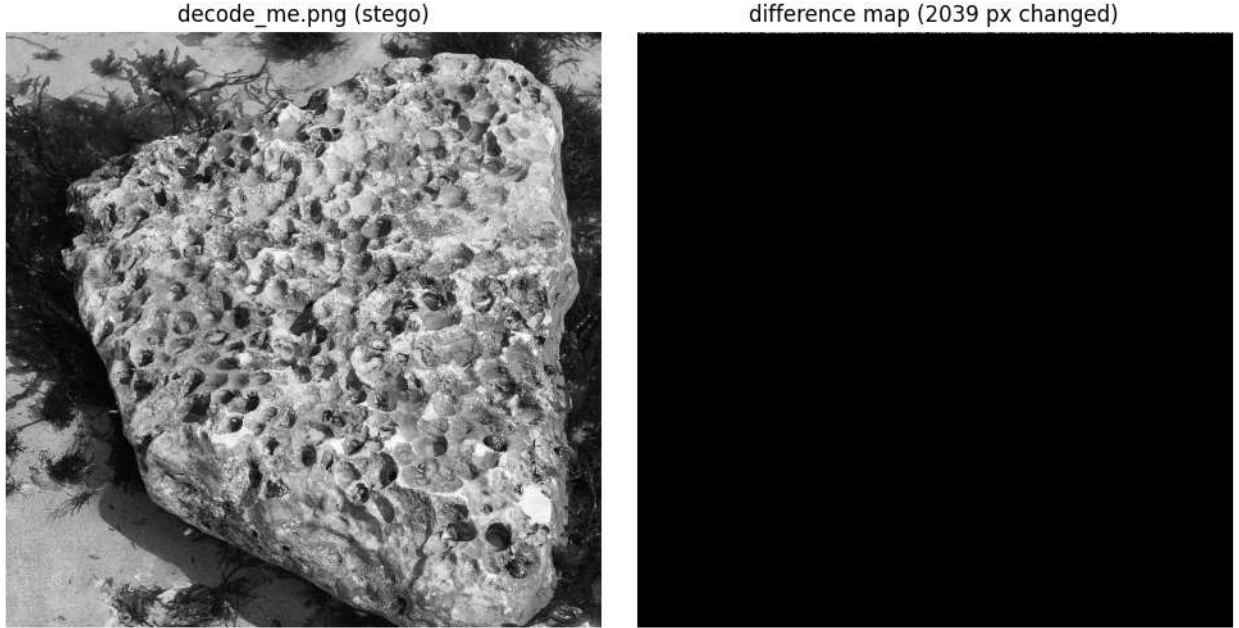


Figure 11: Clean textured cover, LSB stego image, and amplified difference map.

The measured PSNR between the clean cover and the stego image is

$$\boxed{\text{PSNR} = 75.24 \text{ dB}}.$$

Since LSB embedding changes an individual pixel by at most one grey level, the resulting distortion is very small and is visually difficult to distinguish from the original cover.

Textured versus smooth covers. The same payload was embedded at identical pixel locations in both the textured and smooth covers. This allows the effect of local image structure on steganographic detectability to be examined independently of payload size and embedding locations.

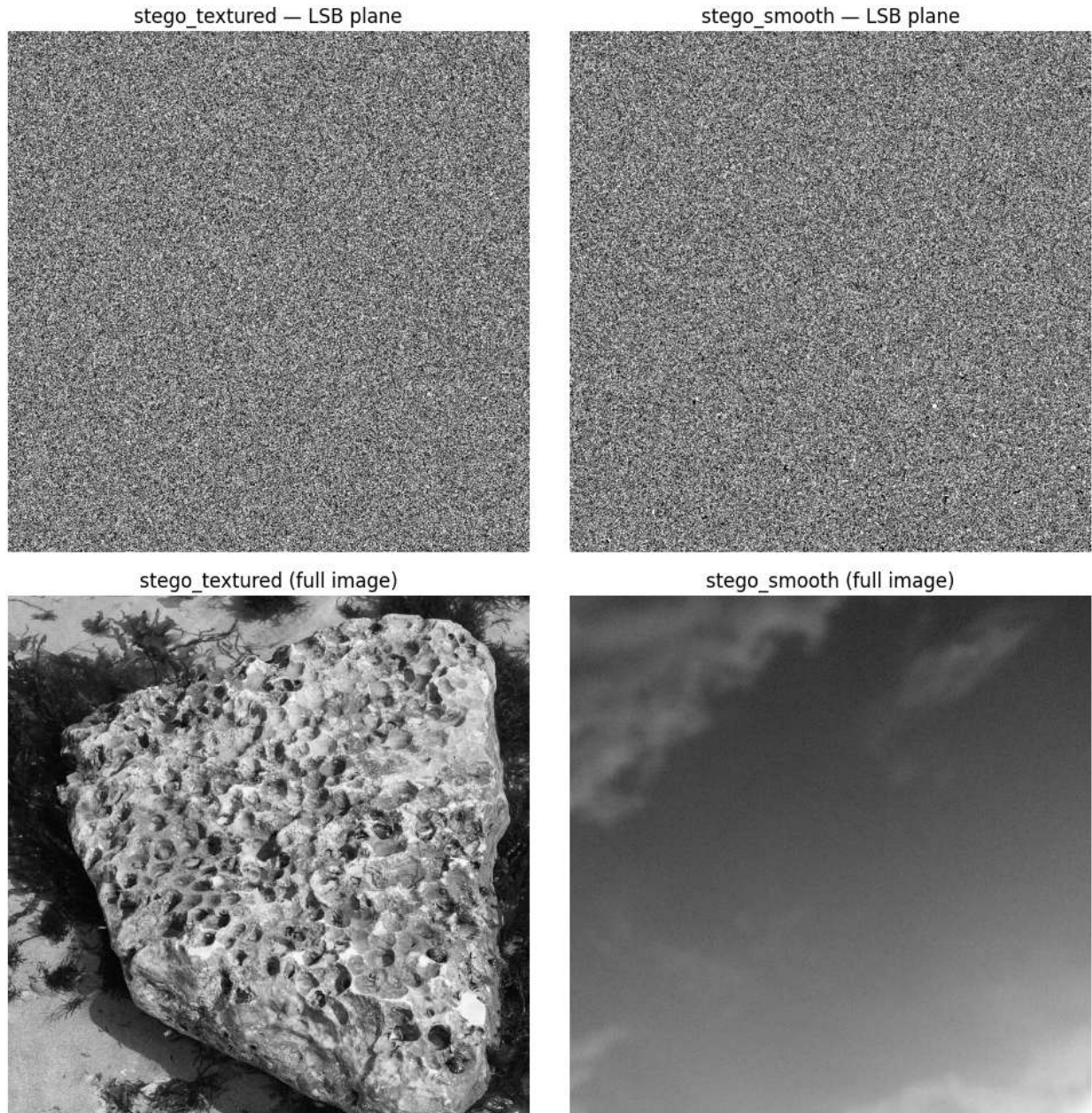


Figure 12: Comparison of textured and smooth covers used for LSB embedding.

For the larger payload experiment, the measured PSNR values were

$$\text{PSNR}_{\text{textured}} = 63.2034 \text{ dB},$$

and

$$\text{PSNR}_{\text{smooth}} = 63.1881 \text{ dB}.$$

The two PSNR values are almost identical. This demonstrates that PSNR alone is insufficient for determining which cover is better for steganographic hiding.

The LSB statistical measurements provide additional evidence. The relative change in the measured chi-square statistic was approximately

$$2.3\%$$

for the textured cover, compared with

$$11.9\%$$

for the smooth cover.

Thus, although the pixel-wise distortion is nearly the same, the embedding is more statistically visible in the smooth image. The textured cover provides better masking because its natural local variation makes the small LSB modifications less distinguishable from existing image structure.

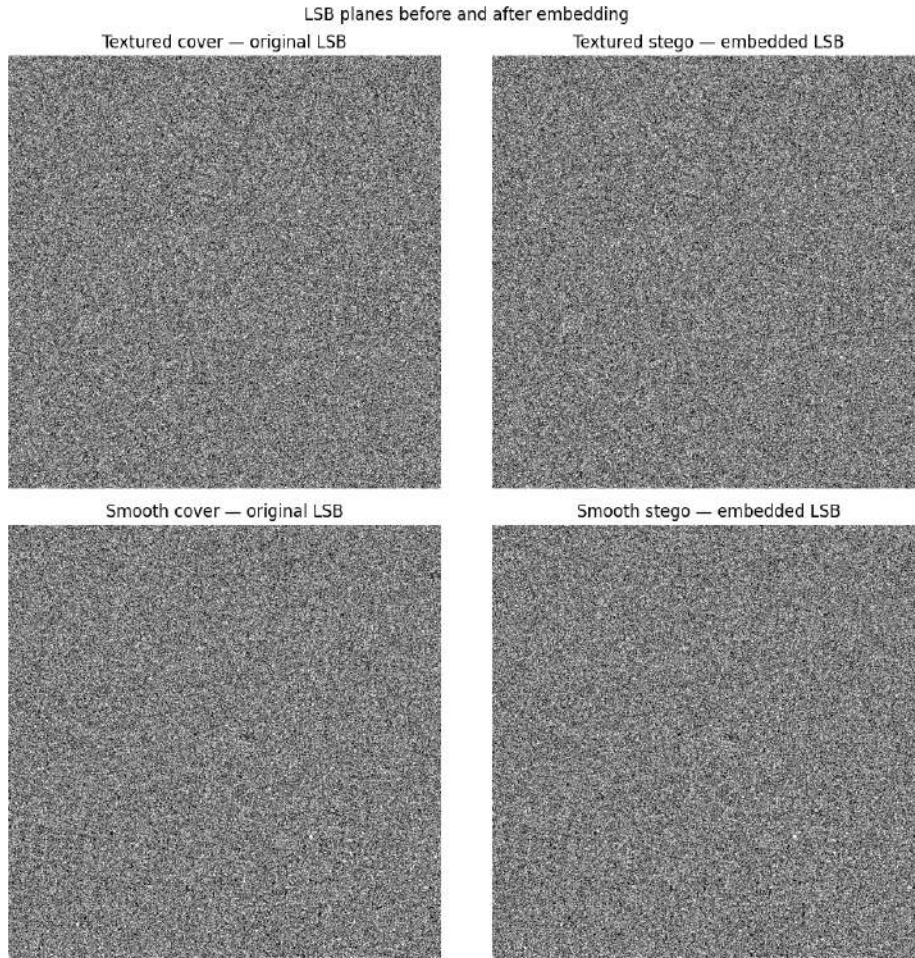


Figure 13: Comparison of the LSB planes before and after embedding for textured and smooth covers.

The corresponding difference maps are shown below.

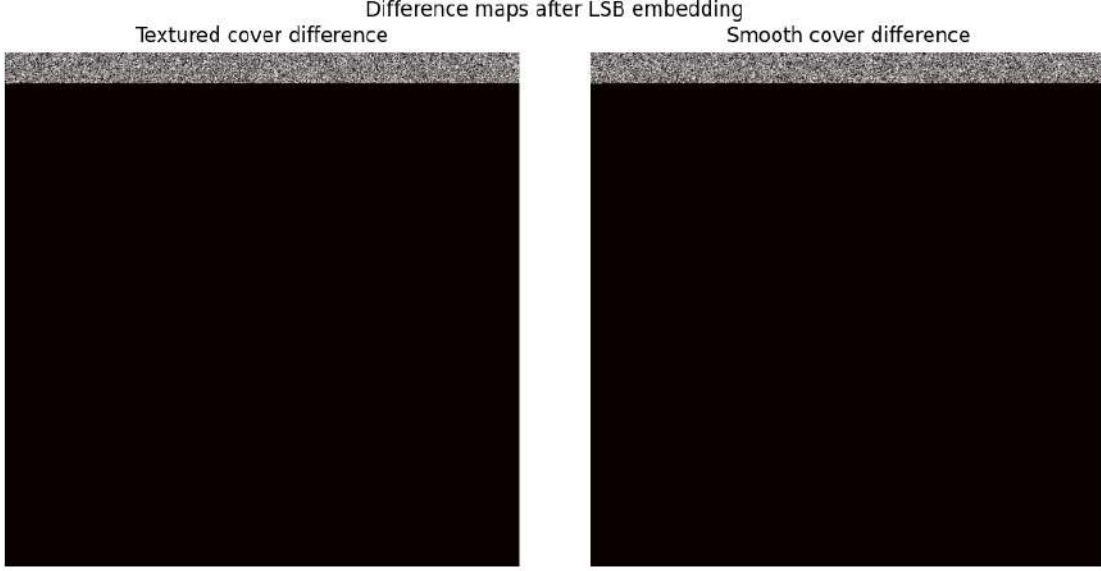


Figure 14: Difference maps for LSB embedding in the textured and smooth covers.

2.3 Robust Steganography

Although conventional LSB embedding has high capacity and very low distortion, it is highly sensitive to channel noise and lossy compression. A robust alternative was therefore implemented using block-based intensity modulation.

The image is divided into 32×32 blocks. One payload bit is embedded into each block by shifting the pixel intensities by

$$\Delta = \begin{cases} +2, & b = 1, \\ -2, & b = 0. \end{cases}$$

Thus each payload bit is represented redundantly over

$$32 \times 32 = 1024$$

pixels.

The decoder is **cover-assisted**. For each block, the received block is compared with the corresponding block in the clean cover. If the mean difference is positive, the bit is decoded as 1; otherwise it is decoded as 0.

For block i ,

$$d_i = \text{mean}(S_i - C_i),$$

and

$$\hat{b}_i = \begin{cases} 1, & d_i > 0, \\ 0, & d_i \leq 0. \end{cases}$$

The use of the clean cover makes the decoder cover-assisted rather than blind.

Imperceptibility. The measured PSNR of the robust stego image was

$$\boxed{\text{PSNR}_{\text{robust}} = 51.14 \text{ dB}}.$$

This exceeds the required 40 dB imperceptibility threshold by

$$51.14 - 40 = 11.14 \text{ dB}.$$

Only 128 blocks are used for embedding. Therefore the number of modified pixels is

$$128 \times 32 \times 32 = 131072.$$

For the 1024×1024 cover, the modified-pixel fraction is

$$\frac{131072}{1024^2} = 0.125.$$

The scheme therefore introduces a spatially distributed but relatively small modification while gaining substantial redundancy.

Gaussian-noise robustness. The robust and naive LSB schemes were tested under the supplied additive Gaussian channel for

$$\sigma \in \{0, 1, 2, 5, 10, 20\}.$$

The measured BER values for the naive LSB method were approximately

σ	Naive LSB BER
0	0.0000
1	0.5625
2	0.5781
5	0.5625
10	0.5000
20	0.5156

Table 4: Naive LSB BER under additive Gaussian noise.

The naive LSB BER rapidly approaches the random-guessing value of approximately 0.5 because the embedded information is stored in individual pixel LSBs.

In contrast, the robust block-based method achieved zero BER over the tested Gaussian noise levels, including the required operating point

$$\boxed{\text{BER}_{\text{robust}}(\sigma = 5) = 0}.$$

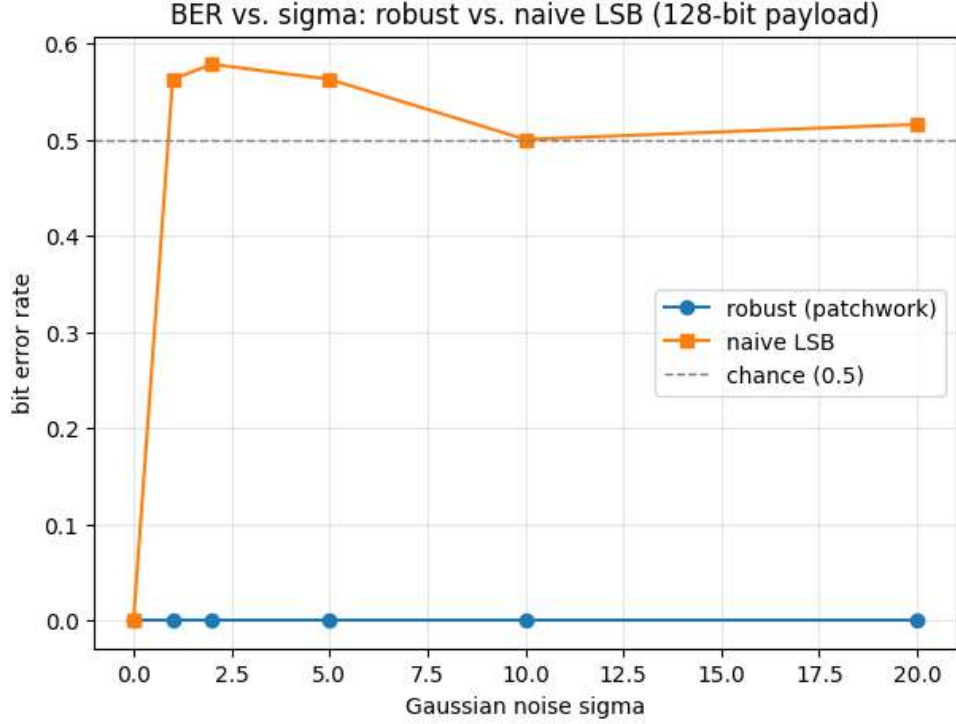


Figure 15: BER versus Gaussian noise level for the robust block-based scheme and naive LSB embedding.

The robustness results are explained by block averaging. Since one bit is distributed across 1024 pixels, independent pixel noise is reduced when the block mean is calculated. The standard deviation of the mean for independent Gaussian noise is approximately

$$\sigma_{\text{mean}} = \frac{\sigma}{\sqrt{1024}} = \frac{\sigma}{32}.$$

At $\sigma = 5$,

$$\sigma_{\text{mean}} = \frac{5}{32} \approx 0.156,$$

which is substantially smaller than the modulation magnitude of two grey levels.

JPEG robustness. The robust and naive methods were also evaluated after JPEG compression with quality factor 75.

The robust decoder achieved

$$\boxed{\text{BER}_{\text{robust, JPEG 75}} = 0}.$$

This satisfies the required zero-error condition.

The naive LSB representation is not expected to survive JPEG reliably because JPEG compression operates on transformed image blocks and does not preserve individual pixel LSBs. The robust scheme instead encodes the payload as a spatially distributed intensity

shift, which survives the compression process sufficiently well for the block-mean detector to recover the embedded bits.

Capacity, robustness and imperceptibility trade-off. The robust scheme embeds 128 bits into a 1024×1024 image. Its payload rate is therefore

$$R = \frac{128}{1024^2} = 1.22 \times 10^{-4} \text{ bits/pixel.}$$

This is much lower than the capacity of direct LSB embedding. The loss in capacity is intentional: each bit is spread over 1024 pixels, providing substantial redundancy against noise and compression.

The resulting design therefore makes the following trade-off:

- LSB embedding provides high capacity and very low visual distortion, but is fragile under noise and JPEG compression.
- Block-based modulation provides much greater robustness by distributing each bit spatially, but has substantially lower capacity.
- The measured robust PSNR of 51.14 dB remains above the required 40 dB threshold, so the reduction in capacity is achieved without introducing excessive visual distortion.
- The decoder is cover-assisted, since the clean cover is required to determine the sign of the block-wise intensity change.

Therefore, the proposed method prioritizes robustness over payload capacity while maintaining acceptable imperceptibility.

Summary

The bit-plane experiments show that the higher-order planes contain most of the visually significant image information, while progressively adding lower-order planes improves reconstruction PSNR by approximately 6 dB per additional bit.

LSB embedding demonstrates that a binary payload can be hidden with very small visual distortion. The measured PSNR of 75.24 dB confirms the high imperceptibility of the basic LSB method. However, comparison of textured and smooth covers shows that PSNR alone does not capture steganographic detectability: the textured cover exhibits a much smaller statistical change under the same embedding operation.

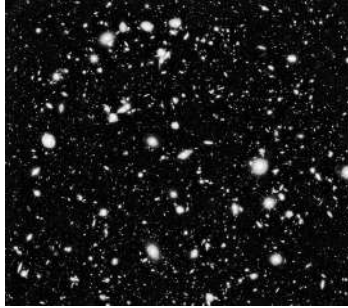
Finally, the robust block-based scheme trades payload capacity for channel robustness. Each bit is distributed over a 32×32 block and decoded using the clean cover. The resulting PSNR is 51.14 dB, while the measured BER is zero at the required Gaussian noise level $\sigma = 5$ and after JPEG compression at quality 75. This demonstrates that spatial redundancy provides substantially greater robustness than individual-pixel LSB embedding.

3 Piecewise linear transforms

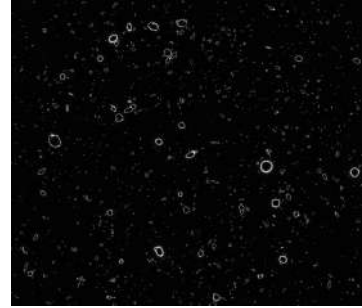
Data



(a) Input: Field



(b) Output A



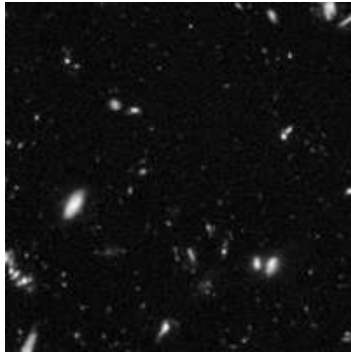
(c) Output B



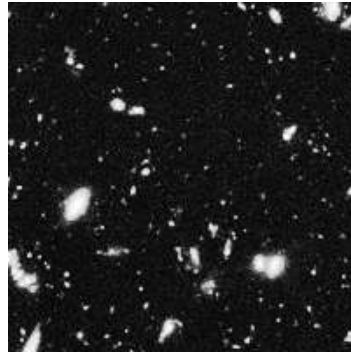
(d) Input: Nebula



(e) Output C



(f) Input: Field Skycrop



(g) Output D

Figure 16: Input images and their corresponding transformed outputs.

3.1 Recover Two Known Transforms

Transform A – Levels Operation

Transform A corresponds to a levels operation that flattens the sky floor, stretches the faint nebulosity, and rolls off the highlights. The input-output pixel pairs were used to recover a piecewise-linear approximation of the grey-level transfer function.

The recovered transfer function contains three linear segments with breakpoints at 12 and 60. The estimated parameters are shown below.

Segment	Input range	Slope	Intercept
1	$0 \leq x \leq 12$	0.0000	0.0000
2	$12 < x \leq 60$	3.9777	-48.6245
3	$60 < x \leq 255$	0.3333	169.6688

Table 5: Recovered parameters for Transform A.

Thus, the recovered transformation is

$$T_A(x) = \begin{cases} 0.0000x + 0.0000, & 0 \leq x \leq 12, \\ 3.9777x - 48.6245, & 12 < x \leq 60, \\ 0.3333x + 169.6688, & 60 < x \leq 255. \end{cases}$$

The measured input-output pixel scatter together with the recovered piecewise-linear transfer curve is shown in Fig. 17.

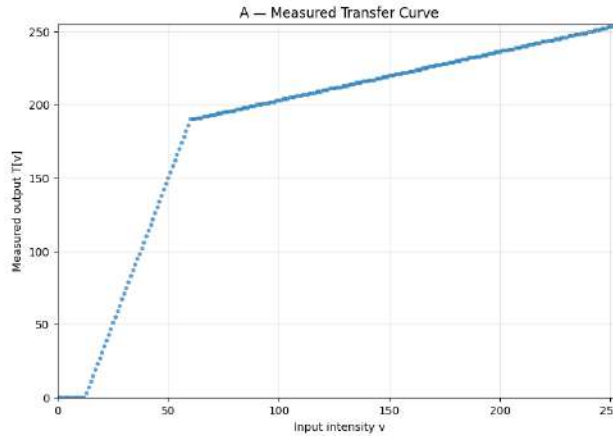


Figure 17: Measured input-output pixel scatter and recovered piecewise-linear fit for Transform A.

The recovered transform was then applied to the original input image. The reconstructed output was compared pixel-by-pixel with the true output. The maximum absolute error was 1, while the mean absolute error was 0.005822.

Transform B – Grey-Level Slicing

Transform B is a grey-level slicing operation and is therefore non-monotonic. The input-output pixel correspondence was used to recover a piecewise-linear approximation of the transformation.

The recovered transformation contains four segments with breakpoints at 69, 109, and 150. The estimated parameters are shown below.

Segment	Input range	Slope	Intercept
1	$0 \leq x \leq 69$	0.2855	-0.4259
2	$69 < x \leq 109$	5.3729	-356.5005
3	$109 < x \leq 150$	-5.3746	825.7753
4	$150 < x \leq 255$	-0.1903	48.0509

Table 6: Recovered parameters for Transform B.

Thus, the recovered transformation is

$$T_B(x) = \begin{cases} 0.2855x - 0.4259, & 0 \leq x \leq 69, \\ 5.3729x - 356.5005, & 69 < x \leq 109, \\ -5.3746x + 825.7753, & 109 < x \leq 150, \\ -0.1903x + 48.0509, & 150 < x \leq 255. \end{cases}$$

The negative slopes in the third and fourth segments demonstrate the non-monotonic nature of the recovered grey-level mapping.

The measured input-output pixel scatter together with the recovered piecewise-linear transfer curve is shown in Fig. 18.

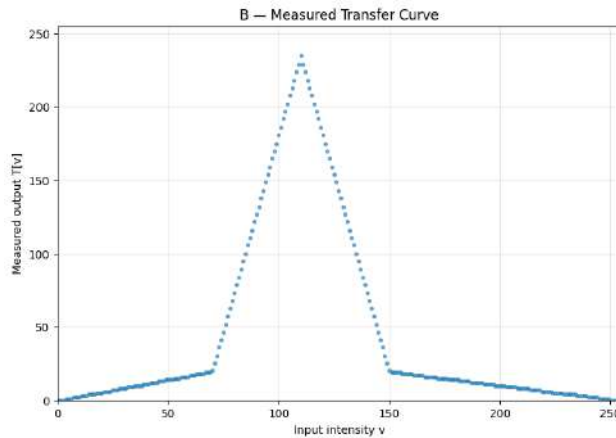


Figure 18: Measured input-output pixel scatter and recovered piecewise-linear fit for Transform B.

The recovered transformation was applied to the original input image and the result

was compared with the true output. The maximum absolute error was 11, while the mean absolute error was 0.005494.

Re-application of the Recovered Transforms

The recovered transformations were re-applied to the original input images. Figure 19 compares the original inputs, the true outputs, and the outputs obtained using the recovered transformations.

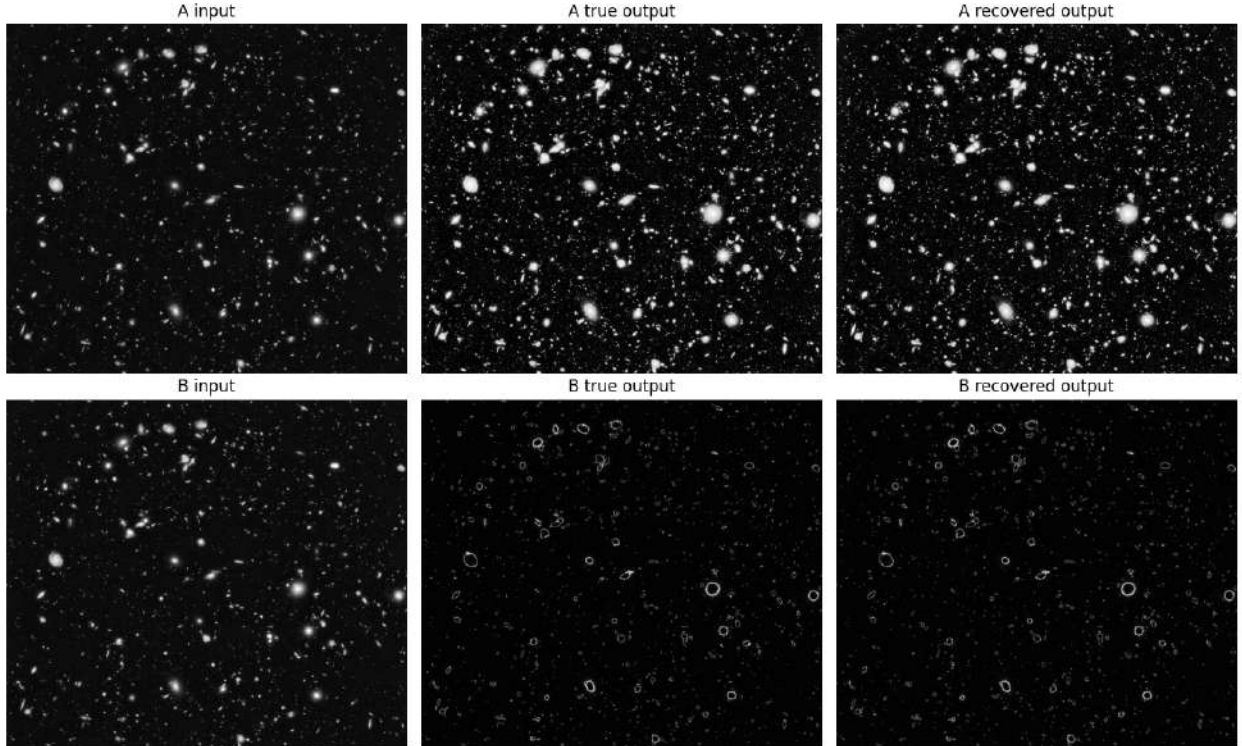


Figure 19: Comparison of the original inputs, true transformed outputs, and outputs obtained by re-applying the recovered transformations for Transform A and Transform B.

Transform	Maximum Absolute Error	Mean Absolute Error
A	1	0.005822
B	11	0.005494

Table 7: Reconstruction errors after re-applying the recovered transformations.

Invertibility. Transform B is recoverable from the input-output pair because the pair reveals the mapping from input to output grey levels, but it is not invertible from the output alone because its non-monotonic mapping can map multiple input levels to the same output level.

3.2 Choosing the Number of Segments

For the third input-output pair, the number of segments in the piecewise-linear transformation was not known in advance. Simply minimizing the least-squares error is not sufficient for selecting the number of segments, since the residual always decreases as additional segments are introduced.

To account for model complexity, a BIC-like criterion was used. For a model containing k linear segments, each segment contributes two parameters (slope and intercept), giving

$$p = 2k.$$

The model score was calculated as

$$\text{Score}(k) = N \log \left(\max \left(\frac{\text{SSE}_k}{N}, \frac{1}{12} \right) \right) + 2k \log N,$$

where N is the number of pixels, SSE_k is the weighted least-squares residual for a k -segment model, and $1/12$ is the variance of a uniform quantisation error over the interval $[-0.5, 0.5]$. The variance floor prevents the model from treating integer-valued output quantisation as additional structure in the underlying transformation.

The resulting scores for models containing between one and six segments are shown in Table 8.

Number of segments	SSE	Penalized score
1	1.15746×10^9	1.56518×10^7
2	1.26734×10^7	4.09477×10^6
3	1.21608×10^6	-1.90552×10^6
4	2.02279×10^5	-6.36124×10^6
5	1.95011×10^5	-6.36121×10^6
6	1.89255×10^5	-6.36118×10^6

Table 8: Segment-count selection for the third input-output pair.

The penalized score decreases substantially as the model complexity is increased from one to four segments, indicating that the additional segments capture genuine structure in the transformation. However, the score increases slightly when moving from four to five and six segments. Therefore, the selected model contains

$$k = 4 \text{ segments.}$$

Figure 20 shows the penalized model score as a function of the number of segments.

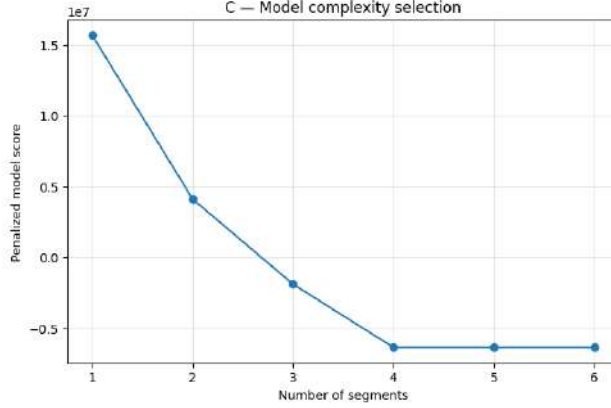


Figure 20: Penalized model score versus number of segments for the third input-output pair. The minimum occurs at four segments.

Re-application of the selected model. The selected four-segment transformation was re-applied to the input image to verify that the recovered model captures the underlying transformation. Figure 21 compares the input image, the true output, the recovered output, and the corresponding absolute error.

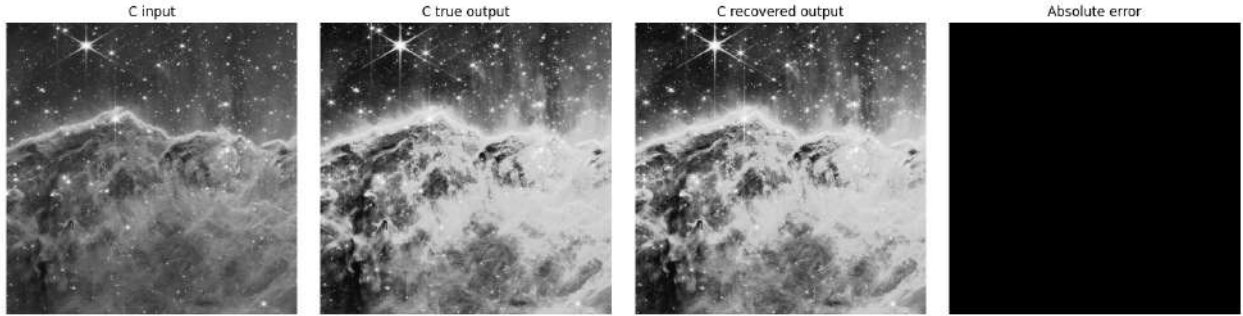


Figure 21: Re-application of the selected four-segment model for Transform C. The recovered output closely matches the true output, with only small quantisation-level differences.

The reconstruction errors are

$$\text{Maximum absolute error} = 2 \quad \text{Mean absolute error} = 0.006389.$$

Avoiding quantisation over-fitting. The quantisation floor provides an explicit stopping criterion once the residual reaches the scale expected from integer-valued output quantisation. Without this floor, adding segments could continue to reduce the residual by fitting the alternating rounding pattern of the lookup table rather than the underlying transform. In the present experiment, the selected model stops at four segments: although the SSE decreases from 2.023×10^5 for four segments to 1.950×10^5 and 1.893×10^5 for five and six segments, respectively, the corresponding penalized scores become worse. Hence, the method does not interpret the small residual improvements as evidence for additional transform segments.

3.3 Identifiability

For the fourth input-output pair, the input image is a 200×200 crop containing predominantly sky. Therefore, only a limited range of input intensity levels is sufficiently represented in the image. Since the transformation is pointwise, $T[v]$ can only be estimated from the data when pixels with input value v are actually present.

(a) Input histogram and support threshold. The input crop contains

$$N = 200 \times 200 = 40000$$

pixels. The input histogram contains 239 of the 256 possible intensity levels, with observed input values ranging from 0 to 246.

To distinguish well-supported intensity levels from isolated or sparsely occurring levels, a level was considered reliable only if it occurred in at least 0.1% of the input pixels. This gives the threshold

$$N_{\text{threshold}} = 0.001 \times 40000 = 40$$

pixels per intensity level.

The 0.1% threshold was chosen as a simple support criterion that removes intensity levels represented by only a small number of pixels. Such levels provide insufficient evidence for estimating $T[v]$ reliably and can otherwise cause the fitted transform to follow sampling noise.

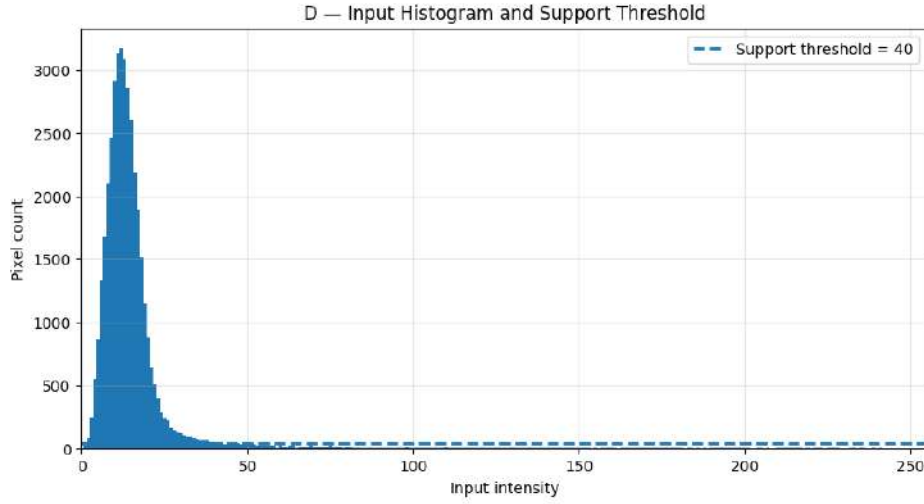


Figure 22: Histogram of the input crop for the fourth pair. The dashed horizontal line indicates the support threshold of 40 pixels.

Using this threshold, the sufficiently supported input levels are

$$\boxed{2-42 \quad \text{and} \quad 47}.$$

The insufficiently supported levels are

0–1, 43–46, 48–255.

Thus, although levels up to 246 occur in the image, most of the higher intensity range is either absent or insufficiently represented to reliably determine the transformation.

(b) Recovering the identifiable part of the transform. The transformation was fitted only using the sufficiently supported input levels. The data supports two linear segments, with a breakpoint at input intensity 20. The recovered parameters are shown in Table 9.

Segment	Input range	Slope	Intercept
1	2–20	1.500809	−0.256315
2	21–47	4.876500	−68.190327

Table 9: Recoverable segments of Transform D.

The recovered transformation over the supported region is therefore

$$T_D(x) = \begin{cases} 1.500809x - 0.256315, & 2 \leq x \leq 20, \\ 4.876500x - 68.190327, & 21 \leq x \leq 47. \end{cases}$$

These are the only segments that can be determined reliably from the available data. The transform is unconstrained over the poorly supported intensity levels, since there are not enough observations to distinguish between different possible values of $T[v]$. The dominant unconstrained region is therefore

$$\boxed{48\text{--}255},$$

with the additional poorly supported ranges 0–1 and 43–46.

(c) Number of reported segments. Only

$$\boxed{2 \text{ segments}}$$

were reported, even though the true transformation contains four segments. This is because the remaining two segments lie outside the well-supported portion of the input intensity distribution. The input-output pair does not contain enough observations in those regions to identify their breakpoints or slopes.

Reporting four segments would therefore imply information that is not contained in the data. In particular, for an input level v that does not occur in the image, $T[v]$ can take any value without changing the observed input-output pairs. Such parameters are fundamentally unidentifiable and cannot be recovered by any fitting method.

The two recovered segments have breakpoint $x = 20$, with slopes 1.500809 and 4.876500, respectively.

4 Histograms

4.1 Global Histogram Equalisation

CDF-based histogram equalisation was implemented from scratch using the normalised inclusive cumulative distribution function. For an input intensity level v , the transformation used was

$$T(v) = \text{round}(255F(v)),$$

where

$$F(v) = \frac{\sum_{u=0}^v h(u)}{N},$$

$h(u)$ is the input histogram and N is the total number of pixels. No $CDF_{\min}$ rescaling was used. The supplied image has dimensions 512×512 , and all 256 input intensity levels occur. The resulting equalised histogram occupies 49 output bins. The implementation therefore follows the specified discrete equalisation rule.

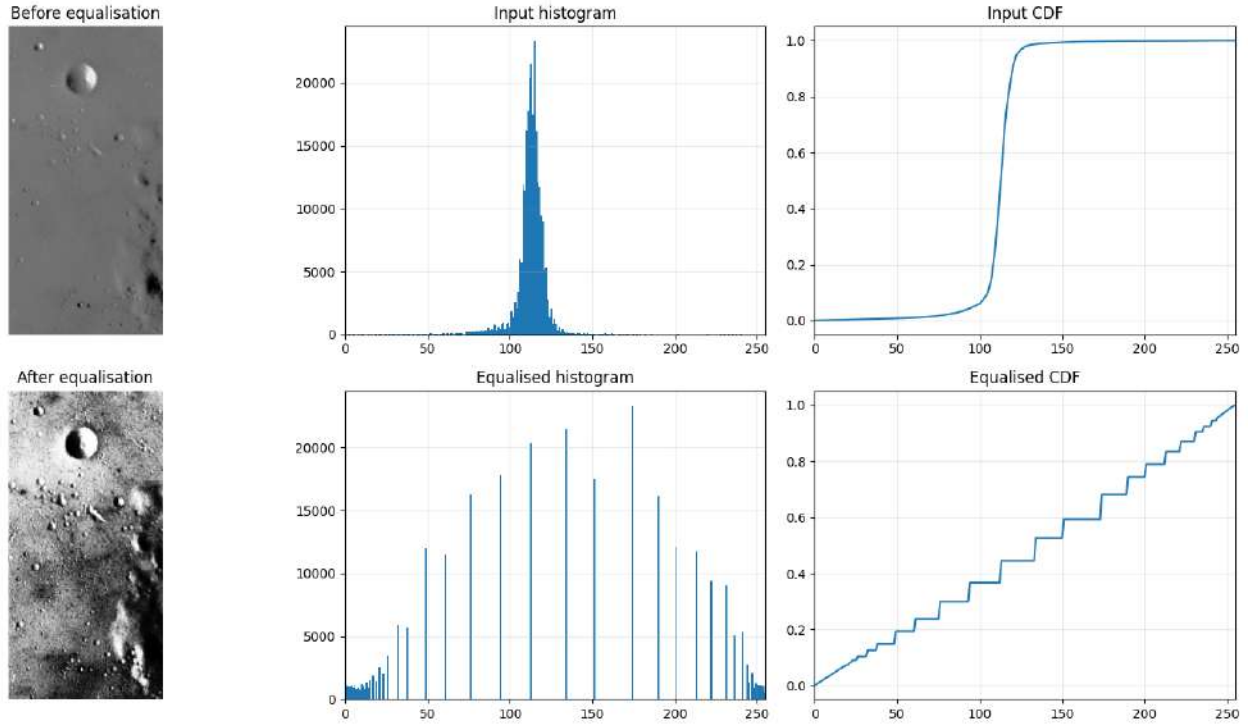


Figure 23: Global histogram equalisation: input image and histogram, input CDF, equalised image and histogram, and equalised CDF.

(a) Why the Equalised Histogram Cannot Be Flat

The equalised histogram is not flat because the mapping is constrained to integer output levels. Let the transformed value of an input level v be

$$y_v = \text{round}(255F(v)).$$

Several consecutive input levels can produce the same value of y_v because the rounding operation maps the continuous CDF values to only 256 discrete output levels. Consequently, multiple input histogram bins can collapse into the same output bin, while some output bins may receive no pixels at all.

More formally, if v_1 and v_2 are consecutive occupied input levels, their CDF values differ by

$$F(v_2) - F(v_1) = \frac{h(v_2)}{N}.$$

There is no requirement that this difference be exactly $1/255$. Therefore, after multiplication by 255 and rounding, the transformed levels need not be distinct or uniformly spaced. The resulting histogram must therefore contain collisions and/or empty bins in general.

For the supplied image, only 49 of the 256 possible output bins are occupied, confirming this discrete-bin effect.

Occupied output bins = 49 < 256

(b) Idempotence of Equalisation

The equalisation operation is idempotent under the specified discrete CDF mapping. Let the occupied input levels be

$$v_1 < v_2 < \cdots < v_m$$

with probabilities

$$p_i = \frac{h(v_i)}{N},$$

and cumulative probabilities

$$F_i = \sum_{j=1}^i p_j.$$

The first equalisation maps every v_i to

$$y_i = \text{round}(255F_i).$$

All pixels belonging to input levels whose rounded CDF values are equal are therefore merged into the same output level. Consider one such output level y . Its cumulative probability in the equalised image is the sum of the probabilities of all original levels mapped to values less than or equal to y . By the definition of rounding, the resulting cumulative probability maps back to the same integer level y under

$$T(y) = \text{round}(255F_Y(y)).$$

Hence every output level produced by the first equalisation is already a fixed point of the equalisation mapping. Applying equalisation again therefore leaves the image unchanged:

$$E(E(I)) = E(I).$$

Thus histogram equalisation is exactly idempotent for this discrete CDF-based transformation.

For the supplied image, the implementation also verifies that the pixel arrays after one and two applications are identical.

$$E(E(I)) = E(I)$$

(c) An Image on Which Equalisation Does Nothing

An image was constructed containing every intensity level from 1 through 255 exactly once. Its histogram is therefore uniform over the occupied intensity range. For this histogram,

$$F(v) = \frac{v}{255},$$

and consequently

$$T(v) = \text{round}\left(255 \frac{v}{255}\right) = \text{round}(v) = v.$$

Therefore,

$$T(v) = v$$

for every occupied intensity level, so equalisation produces no change. The property responsible for this behaviour is the uniform histogram: the CDF is already linear in intensity.

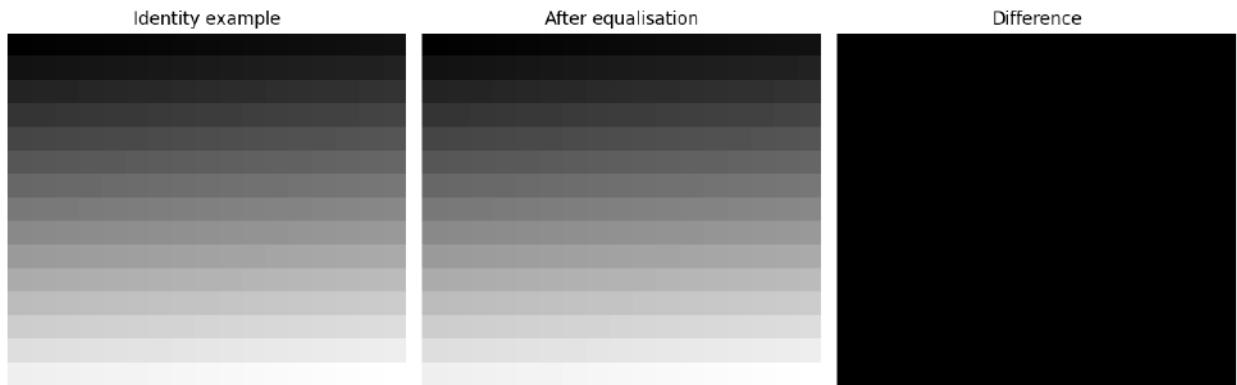


Figure 24: Constructed identity example. The uniform histogram causes equalisation to act as the identity mapping.

4.2 Histogram Matching and Specification

An inverse-CDF histogram specification method was implemented for an arbitrary prescribed target histogram. Given a source CDF F_s and target CDF F_t , each source intensity v is mapped according to

$$T(v) = \min\{z : F_t(z) \geq F_s(v)\}.$$

This allows both image histogram matching and specification to arbitrary target distributions.

(a) Image Histogram Matching

The histogram of `matching/source.png` was matched to the histogram of the supplied reference image. The source image, reference image, resulting image, histograms, and CDFs are shown below.

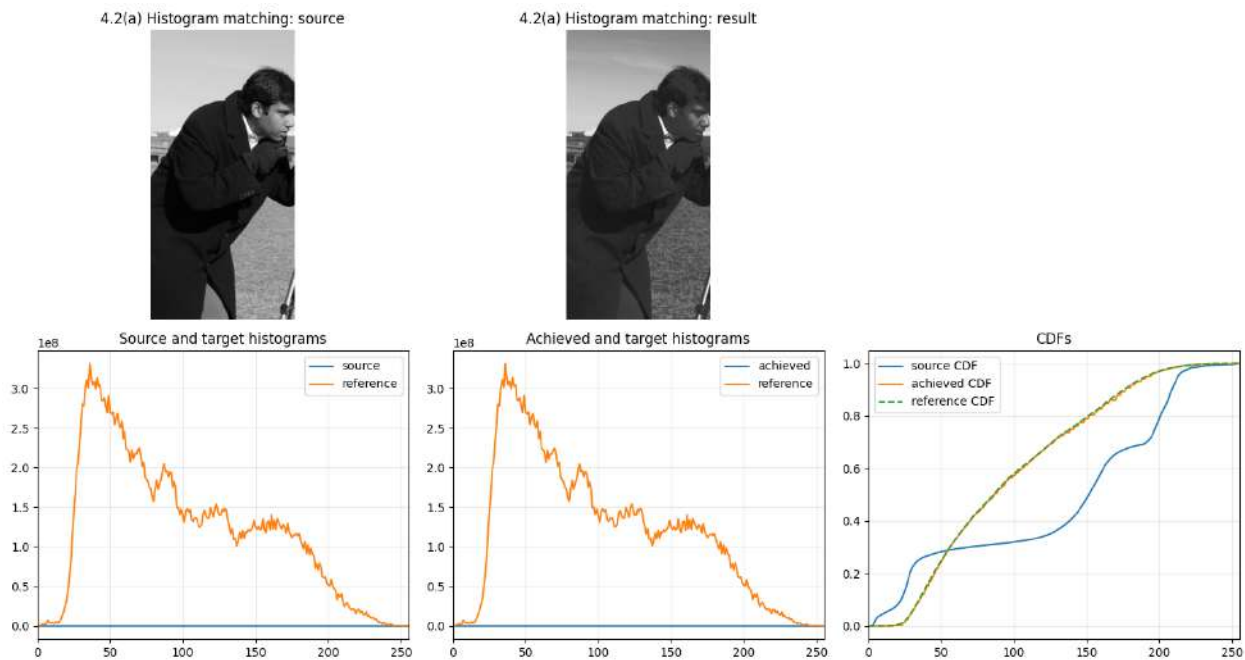


Figure 25: Histogram matching from the supplied source image to the histogram of the supplied reference image.

The Wasserstein-1 distance between the source and reference histograms decreased from

40.774623

to

0.860480

after matching.

(b) Histogram Specification

Two prescribed target distributions were used for `specification/source.png`: a uniform distribution and a Gaussian distribution with

$$\mu = 128, \quad \sigma = 35.$$

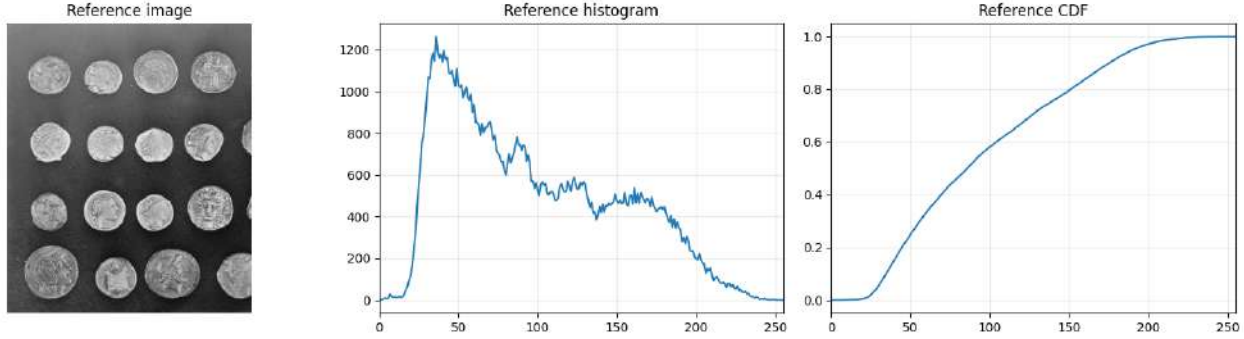


Figure 26: Histogram specification using uniform and Gaussian target distributions.

The Wasserstein-1 distances are summarized in Table 10.

Target	Before	After
Reference	40.774623	0.860480
Uniform	69.146399	1.604801
Gaussian ($\mu = 128, \sigma = 35$)	67.085189	0.767012

Table 10: Wasserstein-1 distances before and after histogram matching/specification.

Uniform specification has the same desired target distribution as the objective of ordinary histogram equalisation. The difference is that histogram specification treats the uniform distribution as an explicit prescribed target and performs inverse-CDF composition. In a finite 8-bit image, both methods can exhibit histogram collisions because the output is restricted to 256 discrete intensity levels.

(c) Colour Histogram Equalisation

The supplied colour image was processed in two ways:

1. independent equalisation of the R, G, and B channels;
2. equalisation of a single BT.601 luminance channel.

The luminance was calculated as

$$Y = 0.299R + 0.587G + 0.114B.$$

The colour results are shown in Fig. 27.



Figure 27: Original colour image, independent RGB-channel equalisation, and BT.601 luminance-only equalisation.

Hue was measured in HSV space using the circular angular distance

$$d(h_1, h_2) = 360 |((h_2 - h_1 + 0.5) \bmod 1) - 0.5|.$$

Pixels were considered valid only when the saturation satisfied

$$S \geq 0.05$$

in both the original and processed images.

The mean hue shift for independent RGB equalisation was

$$\boxed{81.707249^\circ}$$

over 196675 valid pixels, whereas luminance-only equalisation produced

$$\boxed{0.574743^\circ}$$

over 198799 valid pixels.

Independent RGB equalisation is inappropriate because histogram equalisation assumes a scalar ordered intensity coordinate. Applying three independent mappings changes the relative values of the colour channels and therefore changes chromatic direction, causing substantial hue and saturation distortion. In contrast, luminance-only equalisation applies one mapping to brightness while largely preserving the chromatic ratios.

4.3 Local Histogram Processing

The supplied colour image was first converted once to 8-bit BT.601 luminance using

$$Y = \text{round}(0.299R + 0.587G + 0.114B).$$

The resulting luminance image has dimensions 1462×2992 and was used for all subsequent local processing.

(a) Local Histograms

Five spatially distinct regions were selected and their histograms were computed independently.

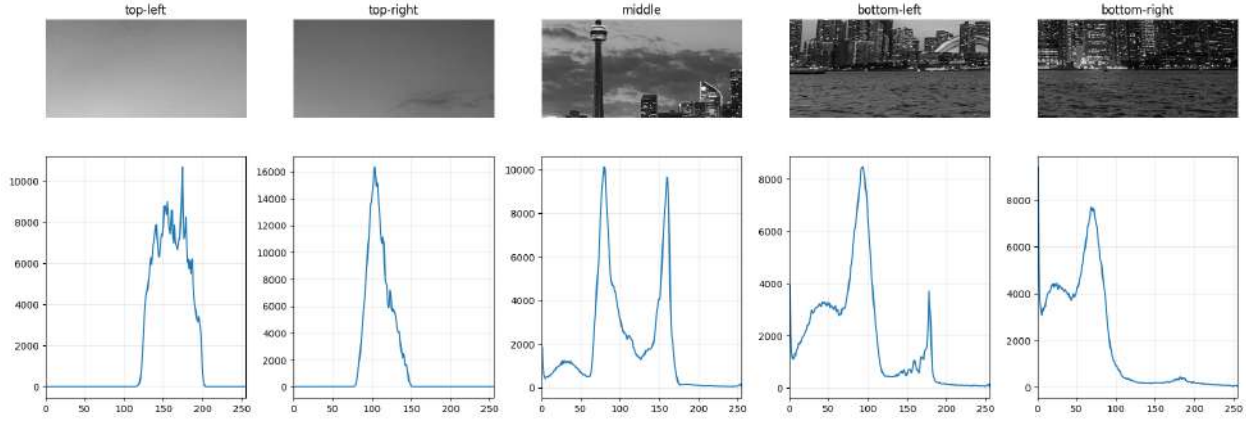


Figure 28: Five distinct local regions and their luminance histograms.

The local histograms differ substantially in their intensity distributions. Therefore, a single global histogram mapping cannot simultaneously provide appropriate local contrast enhancement for all regions. This motivates local histogram processing.

(b) Plain Tiled AHE

Plain tiled adaptive histogram equalisation (AHE) was implemented using an 8×8 tile grid. Each tile was equalised independently, without clipping or interpolation.



Figure 29: Input BT.601 luminance and result of plain tiled AHE using an 8×8 grid.

Two characteristic failure modes are observed:

1. **Tile-boundary seams:** each tile has an independently computed lookup table, so neighbouring tiles can map the same intensity to different output values. This creates visible discontinuities at tile boundaries.
2. **Noise amplification and over-enhancement:** small or nearly uniform tiles have poorly populated histograms. Equalising such distributions can strongly amplify small fluctuations and produce speckle and excessive local contrast.

(c) CLAHE

Contrast-limited adaptive histogram equalisation (CLAHE) was implemented by clipping each tile histogram at a multiple of its mean bin height, redistributing the excess counts, computing a lookup table for each tile, and bilinearly interpolating between the four neighbouring tile mappings at every pixel.

The default configuration used for the displayed result was an 8×8 tile grid with clip limit 3.

(d) Clip-Limit and Tile-Count Ablation

The effect of the clip limit and tile count was evaluated over

$$\text{clip} \in \{1, 2, 3, 10, \infty\}$$

and

$$\text{tiles} \in \{2, 4, 8, 16, 64\}.$$

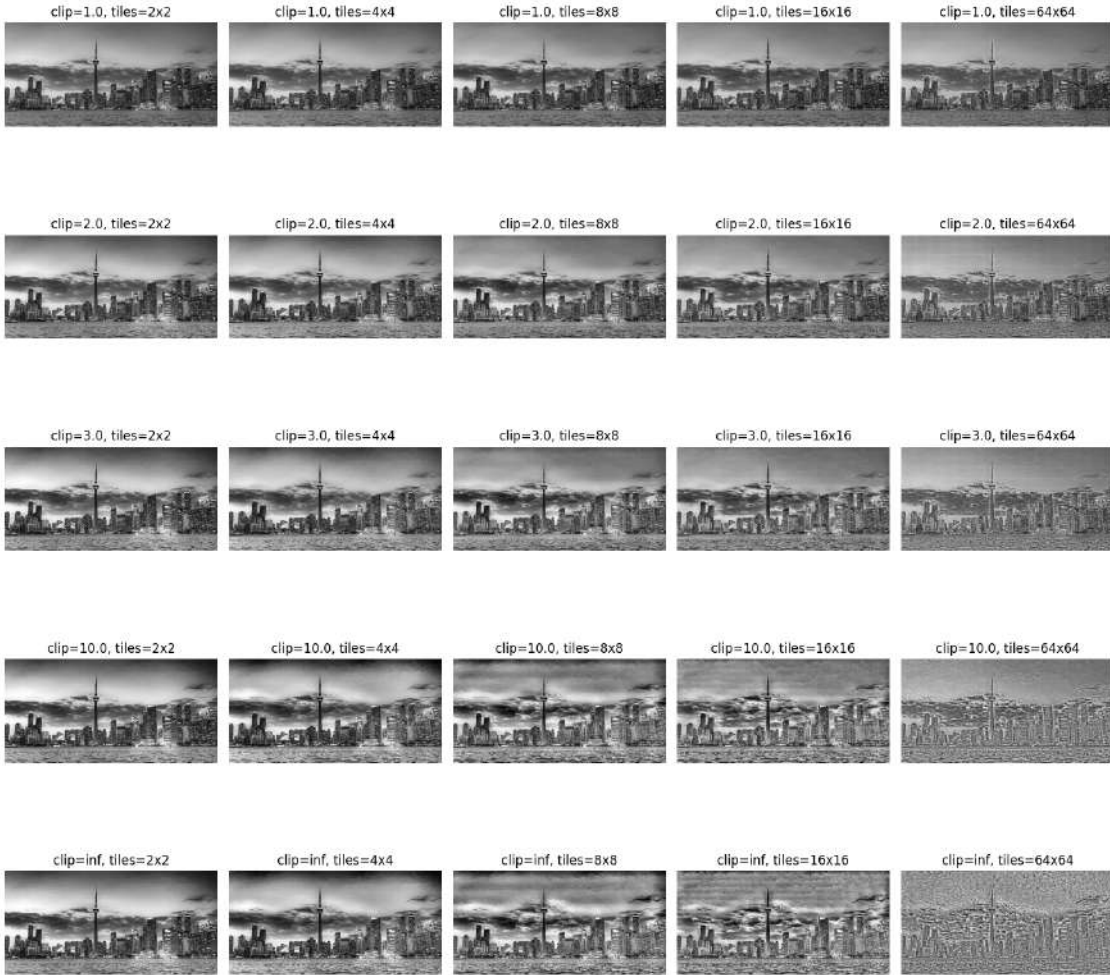


Figure 30: CLAHE ablation over clip limits and tile counts.

At the low clip-limit extreme, $\text{clip} = 1$, strong clipping restricts local contrast enhancement and produces a heavily constrained local histogram. At the opposite extreme, $\text{clip} = \infty$, no histogram clipping occurs, so CLAHE reduces to ordinary tiled AHE and the noise amplification and boundary problems return.

For tile count, a 2×2 grid is close to global processing and therefore does not capture fine local variations. At the opposite extreme, a 64×64 grid produces very small tiles, making the histograms highly sensitive to local noise and producing noisy or block-sensitive results.

(e) Local Histogram Specification

Local histogram specification was implemented by matching each local tile to a prescribed target histogram. The target used here was the global BT.601 luminance histogram of the same image.



Figure 31: Comparison of the original luminance image, CLAHE, and local histogram specification.

The mean Wasserstein-1 distance between each tile’s achieved histogram and the supplied target was

$$W_1^{\text{CLAHE}} = 33.105359$$

for CLAHE, compared with

$$W_1^{\text{spec}} = 2.349362$$

for local histogram specification. Thus, local specification provides a substantially closer match to the prescribed target.

The additional information required is the target histogram itself. Unlike ordinary AHE or CLAHE, which derive the mapping from each tile’s own histogram, histogram specification requires an externally supplied desired distribution.

4.4 Automatic Exposure Correction

The five exposure images were corrected using a single parameter-free function. For each image, BT.601 luminance was computed and the median of the positive luminance values was estimated directly from that image. The luminance was normalised by this median and passed through the fixed compressive tone curve

$$f(x) = \frac{x}{1+x}.$$

The resulting luminance was then used to rescale the RGB channels, preserving their chromatic direction as far as possible. No exposure value or manually selected per-image parameter was supplied.

The corrected exposure set, together with its luminance histograms and CDFs, is shown in Fig. 32.

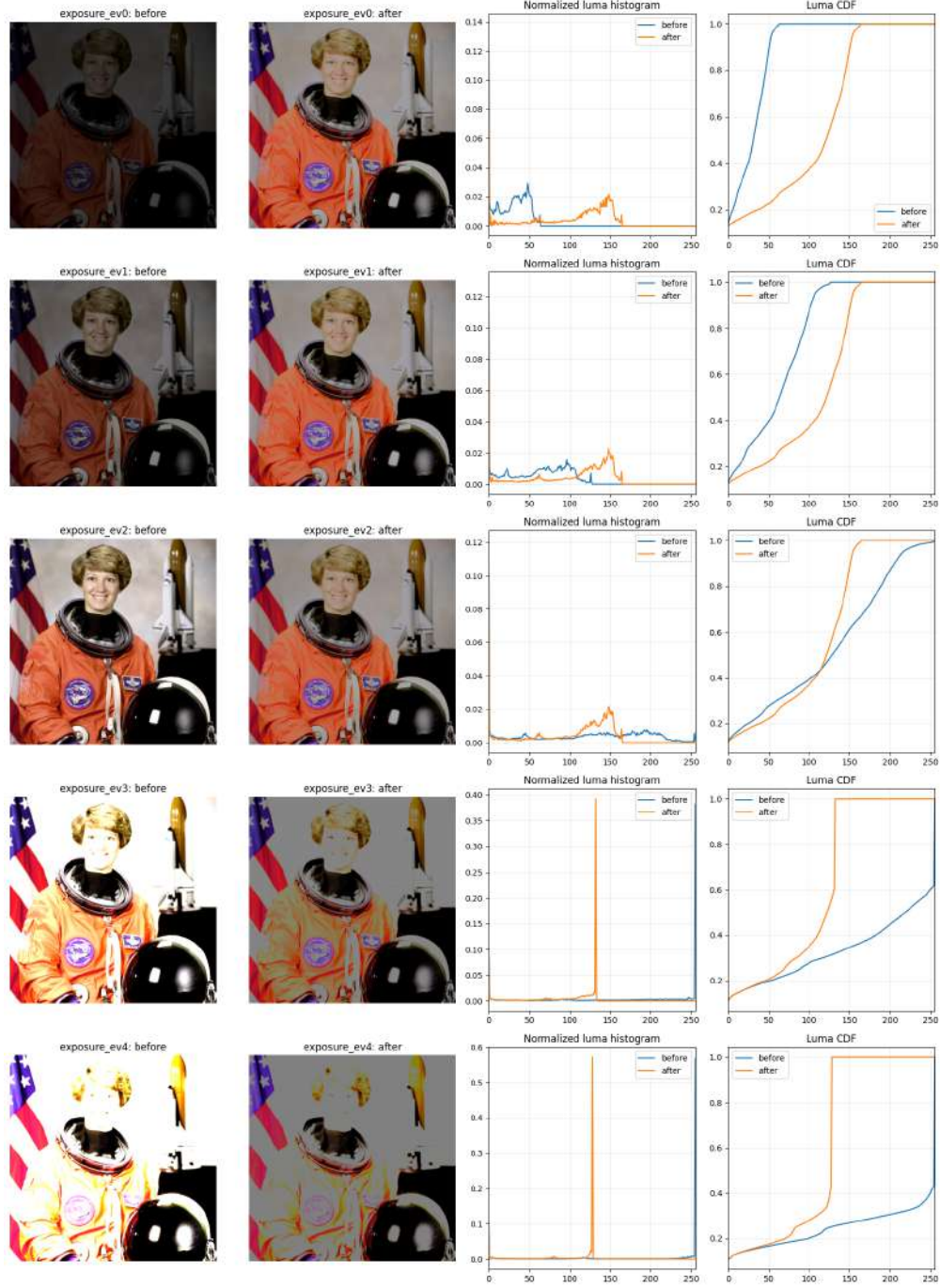


Figure 32: Five exposure images before and after automatic correction, together with their normalised luminance histograms and CDFs.

The ten pairwise Wasserstein-1 distances are shown in Table 11.

Pair	Before	After
EV0–EV1	28.906467	0.604511
EV0–EV2	86.632023	0.800560
EV0–EV3	143.018230	7.892471
EV0–EV4	165.310799	16.205757
EV1–EV2	57.725555	0.221569
EV1–EV3	114.111763	7.621479
EV1–EV4	136.404331	15.972485
EV2–EV3	56.386208	7.430840
EV2–EV4	78.678776	15.786583
EV3–EV4	22.292568	8.395592
Mean	88.946672	8.093185
Maximum	165.310799	16.205757

Table 11: Pairwise Wasserstein-1 distances between the five exposure images before and after automatic correction.

The correction substantially reduces the variation between the exposure histograms. The mean pairwise W1 distance decreases from 88.946672 to 8.093185, while the maximum decreases from 165.310799 to 16.205757.

The corrected luminance statistics are shown in Table 12.

Exposure	Luma standard deviation	1st–99th percentile
EV0	54.123	0–162
EV1	54.020	0–163
EV2	53.880	0–163
EV3	48.172	0–132
EV4	45.699	0–128

Table 12: Luminance statistics of the corrected exposure images.

The non-zero standard deviations and substantial 1st–99th percentile ranges demonstrate that the correction does not collapse the images to a constant or near-constant output.

Failure case. The median-based estimator assumes that the median luminance is representative of the scene exposure. This assumption fails for a scene dominated by a very large black foreground or a very large bright sky, where the median may not represent the desired exposure. The estimator can consequently select an inappropriate normalisation. Furthermore, tone mapping cannot recover information lost through sensor clipping: pixels saturated at the sensor maximum contain no recoverable highlight detail.

5 Green-Screen Compositing

5.1 Naive RGB Keying and Chroma-Space Keying

The video used for this experiment was the ActionVFX practice footage “*Mechanic in Electrical Explosion*”. The clip was selected because it contains a green-screen background together with a foreground subject having fine structures and motion/edge regions, making it suitable for evaluating chroma keying and soft matte extraction. The experiment used frames of resolution 2048×1080 . The loaded sequence contained 60 frames, and the supplied background plate had resolution 5120×1080 , which is $2.5\times$ the frame width and therefore provides sufficient room for horizontal panning.

Naive RGB Green-Dominance Key

A direct RGB key was first implemented using the green-channel dominance measure

$$D(x, y) = G(x, y) - \max(R(x, y), B(x, y)).$$

A pixel was classified as background when

$$D(x, y) > 60.$$

The resulting binary matte assigns $\alpha = 0$ to the estimated green-screen background and $\alpha = 1$ to the foreground.

This approach is simple but is sensitive to illumination, shadows, reflections, and foreground colors. In particular, it assumes that green-screen pixels have a sufficiently large value of $G - \max(R, B)$. A shadowed or unevenly illuminated portion of the green background can therefore have reduced green dominance and be incorrectly classified as foreground. Conversely, foreground regions containing green illumination can be incorrectly classified as background. The limitation arises because the RGB criterion depends directly on channel magnitudes rather than separating chrominance from brightness.



Figure 33: Failure of the naive RGB green-dominance key on the representative frame.

YCbCr Chroma Key

To reduce sensitivity to brightness variation, the RGB image was converted to BT.601 studio-range YCbCr. For normalized RGB values $R, G, B \in [0, 1]$,

$$Y = 16 + 65.481R + 128.553G + 24.966B, \quad (1)$$

$$C_b = 128 - 37.797R - 74.203G + 112B, \quad (2)$$

$$C_r = 128 + 112R - 93.786G - 18.214B. \quad (3)$$

The green-screen key colour was estimated from a 24-pixel border around the representative frame. Taking the median chrominance values of the border gave

$$(C_b^k, C_r^k) = (115.965, 105.443).$$

For each pixel, the chroma distance from this key colour was calculated as

$$d(x, y) = \sqrt{(C_b - C_b^k)^2 + (C_r - C_r^k)^2}.$$

Pixels close to the key colour are therefore identified using chrominance rather than absolute RGB intensity.

5.2 Soft Matte and Spill Suppression

Chroma-Distance Populations

To determine suitable thresholds, three populations were examined: confident backing pixels, boundary pixels, and confident foreground pixels. The resulting chroma-distance statistics are shown in Table 13.

Table 13: Chroma-distance statistics for the representative frame.

Population	Number of pixels	Mean	P5	P95
Backing	302000	7.91	1.03	47.88
Boundary	1432248	1.82	0.14	3.67
Foreground	477592	41.71	5.35	64.76

The backing and foreground populations are not perfectly separated. In particular, the large boundary population indicates significant overlap near the transition between subject and background. Nevertheless, the distributions provide a useful basis for constructing a continuous matte rather than making a hard binary decision.

The thresholds were selected from the measured distributions:

$$t_{\text{in}} = 21.61, \quad t_{\text{out}} = 41.61.$$

These values define the transition region from confidently classified background to confidently classified foreground.

Continuous Alpha Matte

A fractional alpha matte was computed from the chroma distance using

$$\alpha(x, y) = \text{clip} \left(\frac{d(x, y) - t_{\text{in}}}{t_{\text{out}} - t_{\text{in}}}, 0, 1 \right).$$

Thus,

$$\alpha = \begin{cases} 0, & d \leq t_{\text{in}}, \\ \frac{d - t_{\text{in}}}{t_{\text{out}} - t_{\text{in}}}, & t_{\text{in}} < d < t_{\text{out}}, \\ 1, & d \geq t_{\text{out}}. \end{cases}$$

Approximately

7.60%

of the image pixels were assigned fractional alpha values, corresponding primarily to subject boundaries, hair, motion-blurred regions, and partially transparent regions.

The final composite was produced using the standard alpha-compositing equation

$$I = \alpha F + (1 - \alpha)B,$$

where F is the foreground image and B is the replacement background.



Figure 34: Representative frame and the recovered continuous alpha matte.

Spill Suppression

Green spill was reduced before alpha compositing. The green excess was defined as

$$E_G = \max \left(G - \frac{R + B}{2}, 0 \right).$$

The correction was scaled using the factor $(1 - \alpha)$ so that pixels close to the background receive stronger correction while confidently foreground pixels remain largely unchanged. This gives a spatially continuous correction across the matte boundary.

The mean residual green excess measured only over fractional-alpha pixels decreased from

0.60 to 0.07.

Thus, the measured green excess was reduced by approximately 88.3%.



Figure 35: Compositing without and with green-spill suppression.

The use of $(1 - \alpha)$ is appropriate because spill is strongest near partially keyed regions, while applying the same correction to fully foreground pixels would unnecessarily alter their original colour.

Naive versus Soft Matte

Because ground-truth alpha values are not available for the footage, the naive RGB matte was compared directly with the chroma-based soft matte. The mean absolute difference was

$$\text{MAE} = \text{mean} |\alpha_{\text{naive}} - \alpha_{\text{soft}}| = 0.8238.$$

The corresponding gradient-field disagreement was

$$9.2652.$$

The large matte difference demonstrates that the two methods make substantially different decisions, particularly near boundaries. The soft matte preserves gradual transitions rather than forcing each pixel to either foreground or background, making it better suited to hair, motion blur, and partially transparent regions. The gradient comparison is particularly relevant because matte gradients determine the sharpness and smoothness of the final composite boundary.



Figure 36: Naive matte, soft matte, and their absolute difference.

5.3 Vectorised Video Compositing

For video processing, the chroma-to-alpha relationship was accelerated using a 128×128 lookup table over the (C_b, C_r) space. The LUT converts the chroma-distance calculation into a table lookup for each pixel.

The maximum absolute difference between the LUT-derived alpha and the original continuous chroma computation on the representative frame was

$$\max |\alpha_{\text{soft}} - \alpha_{\text{LUT}}| = 0.06692.$$

The 60 frames were composited over the wide background plate using a linearly varying horizontal crop, producing a panning background. The total processing time was

$$35.585 \text{ s},$$

corresponding to an effective processing speed of

$$1.7 \text{ fps}.$$

The timing breakdown is shown in Table 14.

Table 14: Video compositing runtime breakdown.

Stage	Time (s)	Fraction
Panning crop/resize	0.289	0.8%
LUT keying	18.187	51.1%
Spill suppression	8.305	23.3%
Alpha blending	8.797	24.7%
Total	35.585	100%

The dominant bottleneck is LUT keying, which accounts for 51.1% of the measured processing time. Although the LUT removes the expensive distance computation, the YCbCr conversion and associated array operations still dominate the implementation.

The resulting sequence was encoded as an H.264 MP4 video at 24 fps.

Temporal Flicker Reduction

Per-frame keying can produce temporal instability because small frame-to-frame chroma variations cause the estimated alpha to change even when the underlying scene is static. Temporal smoothing was therefore applied to the alpha stack using a centred moving-average filter with a window of nine frames.

Flicker was measured from the second temporal difference of alpha values in a static image region. Before temporal smoothing,

$$F_{\text{before}} = 0.00241.$$

After the nine-frame temporal filter,

$$F_{\text{after}} = 0.00027.$$

The resulting reduction was

$$\frac{0.00241 - 0.00027}{0.00241} \times 100 = 88.9\%.$$

Thus, temporal filtering substantially reduces frame-to-frame matte fluctuations in the measured static region.

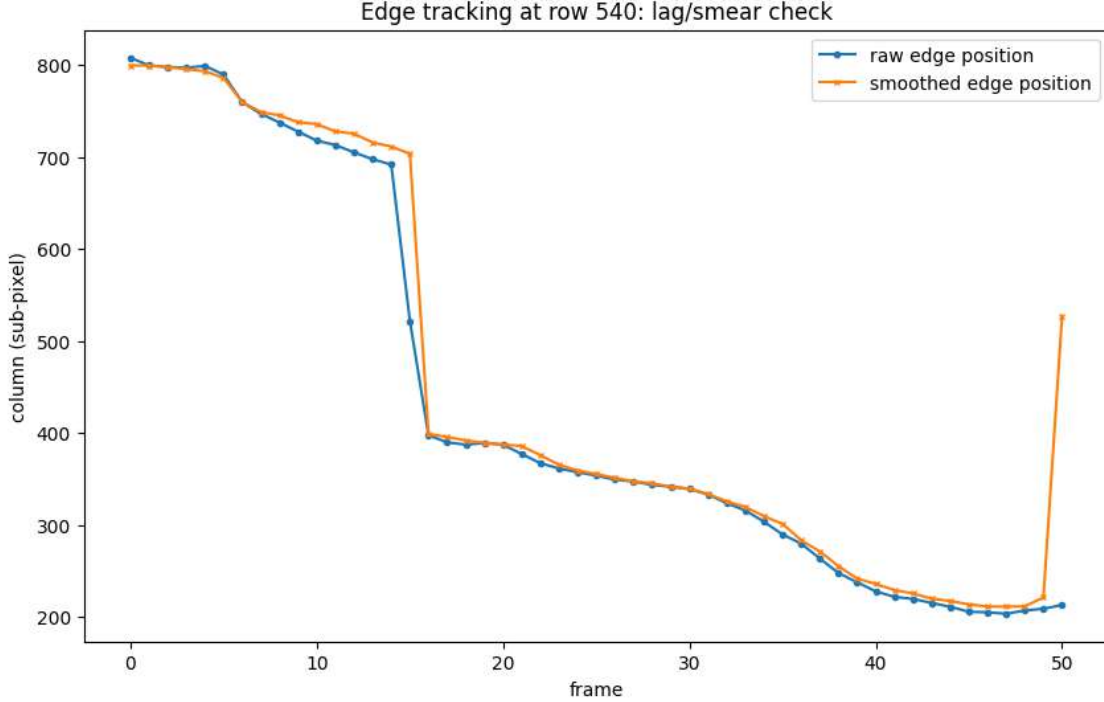


Figure 37: Temporal flicker before and after alpha smoothing.

To check that temporal smoothing does not introduce excessive lag or smear on moving boundaries, the 0.5-alpha crossing position was tracked along a representative row through the moving subject. The raw and temporally smoothed edge trajectories were compared over the sequence. This provides a direct check that the reduction in flicker is not obtained at the cost of a visibly delayed moving edge.

5.4 Discussion

The experiment demonstrates the limitations of simple RGB green dominance and the advantages of chroma-based soft keying. RGB thresholding is highly dependent on absolute channel intensities and can therefore fail under shadows and illumination changes. In contrast, YCbCr provides a more useful representation because the chromatic information is represented separately from luminance.

The continuous alpha matte is especially important for realistic compositing. Approximately 7.60% of the representative frame required fractional alpha values, indicating that a hard foreground/background decision would discard substantial transition information. The measured matte difference of 0.8238 relative to the naive binary key further shows that the two methods behave very differently around the subject boundary.

Green-spill suppression also produced a substantial improvement, reducing the measured mean green excess from 0.60 to 0.07. Finally, temporal smoothing reduced measured flicker by 88.9%, while edge tracking was used to verify that moving boundaries remained stable.

The final pipeline therefore consists of YCbCr conversion, chroma-distance estimation, continuous alpha generation, green-spill suppression, alpha compositing onto a panning background, and temporal alpha smoothing. The current implementation processes the complete sequence at approximately 1.7 fps, with LUT-based keying identified as the primary computational bottleneck.